



Módulo: Acceso a datos

UT1: Manejo de ficheros y flujos de datos

Código 0486

Nivel **Ciclo Formativo de Grado Superior**
Ciclo **Desarrollo de Aplicaciones**
Multiplataforma

Andrés Lorenzo Bleda Tomás

Índice

Resultados de aprendizaje y criterios de evaluación de la UT1	4
Glosario de términos	4
1 Introducción a los ficheros y su estructura	5
1.1 Persistencia de datos en ficheros	5
1.2 Clasificación de los ficheros por su contenido	6
1.3 Clasificación de los ficheros por el tipo de acceso	6
1.4 Mapa conceptual	7
2 Clases para gestión de ficheros y directorios	8
2.1 La clase File	8
3 Operaciones sobre ficheros con Java	10
3.1 Operaciones de lectura	11
3.2 Operaciones de escritura	11
4 Formas de acceso a ficheros y flujos de datos	13
4.1 Acceso secuencial (Flujos de datos – STREAM)	13
4.1.1 Clases relacionadas con flujos de datos	13
4.1.2 Clases para recodificación	15
4.1.3 Clases para buffering	16
4.1.4 Operaciones de lectura para flujos de entrada	16
4.1.5 Operaciones de escritura para flujos de salida	17
Ejemplos prácticos de lectura y escritura secuencial	18
4.2 Acceso aleatorio	20
4.2.1 Operaciones con ficheros	20
Ejemplo práctico: registros de longitud fija	22
4.3 Aplicaciones reales e importancia de la gestión de ficheros	23
5 Trabajo con ficheros XML	25
5.1 Conceptos clave	25
5.2 Parser o analizador XML	28
5.3 Vinculación o binding (JAXB)	34
5.4 Conversión de ficheros XML a otros formatos	40
6 Manejo de excepciones	45
6.1 Introducción a las excepciones en Java	45
6.2 Captura y gestión de excepciones	48

6.3	Gestión diferenciada de distintos tipos de excepciones	50
6.4	Declaración de excepciones lanzadas por un método.....	52
7	Documentación de las aplicaciones.....	55
8	Bibliografía	55

Resultados de aprendizaje y criterios de evaluación de la UT1

¿QUÉ DEBEMOS APRENDER DURANTE LA UNIDAD DE TRABAJO Y QUÉ SE VA A EVALUAR?

RESULTADO DE APRENDIZAJE: <u>RA1</u>	
<i>RA1.- Desarrolla aplicaciones que gestionan información almacenada en ficheros identificando el campo de aplicación de los mismos y utilizando clases específicas.</i>	
CRITERIOS DE EVALUACIÓN	
a)	Se han utilizado clases para la gestión de ficheros y directorios.
b)	Se han valorado las ventajas y los inconvenientes de las distintas formas de acceso.
c)	Se han utilizado clases para recuperar información almacenada en ficheros.
d)	Se han utilizado clases para almacenar información en ficheros.
e)	Se han utilizado clases para realizar conversiones entre diferentes formatos de ficheros.
f)	Se han previsto y gestionado las excepciones.
g)	Se han probado y documentado las aplicaciones desarrolladas.

Glosario de términos

Acceso aleatorio: Tipo de acceso a un fichero que permite acceder directamente a los datos situados en cualquier posición del fichero.
Acceso secuencial: Tipo de acceso a un fichero con el que la única manera de acceder a principio hasta dicha posición.
Codificación de texto: Una manera particular de representar una secuencia de caracteres de texto mediante una secuencia de bytes.
Fichero de texto: Fichero que contiene texto.
Fichero: Unidad fundamental de almacenamiento. Consiste en una secuencia de bytes. Con una adecuada organización, se puede utilizar para almacenar cualquier tipo de información.
Índice: Fichero que permite recorrer los contenidos de otro fichero en un orden determinado.
Registro: Estructura para representar información. Consta de una serie de campos, en cada uno de los cuales se puede almacenar un dato particular.

1 Introducción a los ficheros y su estructura

1.1 Persistencia de datos en ficheros

Los **ficheros** son la forma más básica de guardar información. De hecho, por muy sofisticado que sea un sistema, **al final los datos se escriben en ficheros**. Durante décadas (hasta que en los años ochenta tomaron fuerza las bases de datos relacionales) los ficheros fueron el medio principal de almacenamiento.

El término *fichero* viene de los antiguos **archivadores de fichas de papel**: cada ficha compartía la misma estructura (campos fijos como *título, autor, tema*). Esa idea pasó al mundo digital: los ficheros almacenan **registros** con **campos de longitud fija**. Para **acelerar búsquedas**, se añadieron **ficheros índice** que permitían acceder a los registros en un orden determinado. IBM desarrolló el método **ISAM** (*indexed sequential access method*), y lenguajes como **COBOL** (creado en 1959 y aún presente en sectores como la banca) ofrecieron un excelente soporte para estos **ficheros indexados**.

Con el tiempo, las **bases de datos relacionales** fueron sustituyendo a los sistemas basados en ficheros para gestionar grandes volúmenes y estructuras muy regulares. Aun así, es importante entender que **las bases de datos también utilizan internamente sistemas de gestión de ficheros** para almacenar sus datos.

En esta unidad presentaremos **organizaciones sencillas de ficheros** y aprenderemos a escribir **programas en Java** para **consultar, añadir, borrar y modificar** la información que contienen.



LIMITACIONES DE LOS FICHEROS DE TEXTO

- **Consultas complejas**: si hay que relacionar mucha información, programar las aplicaciones resulta difícil.
- **Grandes volúmenes y cambios frecuentes**: el rendimiento cae, sobre todo con **borrados y modificaciones**.
- **Acceso concurrente**: varios programas actualizando a la vez pueden causar **inconsistencias** o incluso **daños en los ficheros**. Controlar accesos y añadir **transacciones** complica mucho el sistema.
- **Integridad y reglas de negocio**: es **difícil** imponer y verificar **restricciones de integridad**.
- **Redundancia**: duplicar datos desperdicia espacio y puede generar **valores distintos** de la misma información en lugares diferentes.

¿Siguen teniendo sentido los ficheros?

Sí. Aunque las bases de datos relacionales han reemplazado a los ficheros en **grandes colecciones con estructura muy regular**, los ficheros siguen siendo clave en muchos casos:

- **Formatos XML** (que veremos más adelante) para almacenar todo tipo de datos.

- **Sistemas de correo electrónico**, que guardan su información en **ficheros con formatos sencillos**.
- **Procesos masivos** (periódicos o puntuales) que consumen **ficheros de texto**.
- **Intercambio de datos** entre sistemas (**exportación e importación**).
- **Copias de seguridad**.
- Y, en general, **aplicaciones** que trabajan con **formatos de fichero muy diversos**.

En resumen, **los ficheros son simples y útiles**, pero tienen **límites claros** de escalabilidad, integridad y concurrencia. Elegir entre **ficheros** y **bases de datos** depende del **tipo de consultas**, el **volumen**, la **frecuencia de cambios** y la **necesidad de consistencia**.

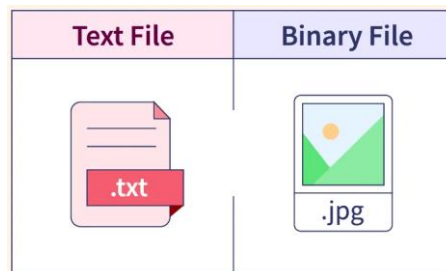
1.2 Clasificación de los ficheros por su contenido

Un **fichero** es una secuencia de bytes; por tanto, puede guardar cualquier información. Se identifica por su **nombre** y su **ubicación** en la jerarquía de directorios.

Según su contenido distinguimos dos tipos:

1. **Ficheros de texto**: contienen solo **caracteres**. Incluyen letras, números, signos de puntuación, espacios y separadores (tabuladores, retornos). Su contenido se puede ver y modificar con **editores de texto** (por ejemplo, Bloc de notas o gedit).
2. **Ficheros binarios**: abarcan el **resto**. Pueden almacenar cualquier tipo de datos y, en general, requieren **programas específicos** para mostrarlos. Los **programas** también se guardan como ficheros binarios.

En resumen, todo son bytes; pero los ficheros podemos clasificarlos como de **texto** o **binarios**, según cómo deban **interpretarse**.



1.3 Clasificación de los ficheros por el tipo de acceso

Según **cómo llegamos a los datos**, distinguimos dos formas de acceso:

- **Acceso secuencial**: se empieza por el **principio del fichero** y se avanza en orden. Para llegar a una posición concreta, hay que pasar por todo lo anterior. Es el modo natural cuando el soporte o el origen de datos **solo permite recorrer** la información de principio a fin. Un ejemplo clásico es la **cinta magnética** (hoy en desuso).

- **Acceso aleatorio:** permite saltar **directamente** a cualquier posición del fichero para **leer o escribir** sin recorrer lo previo.

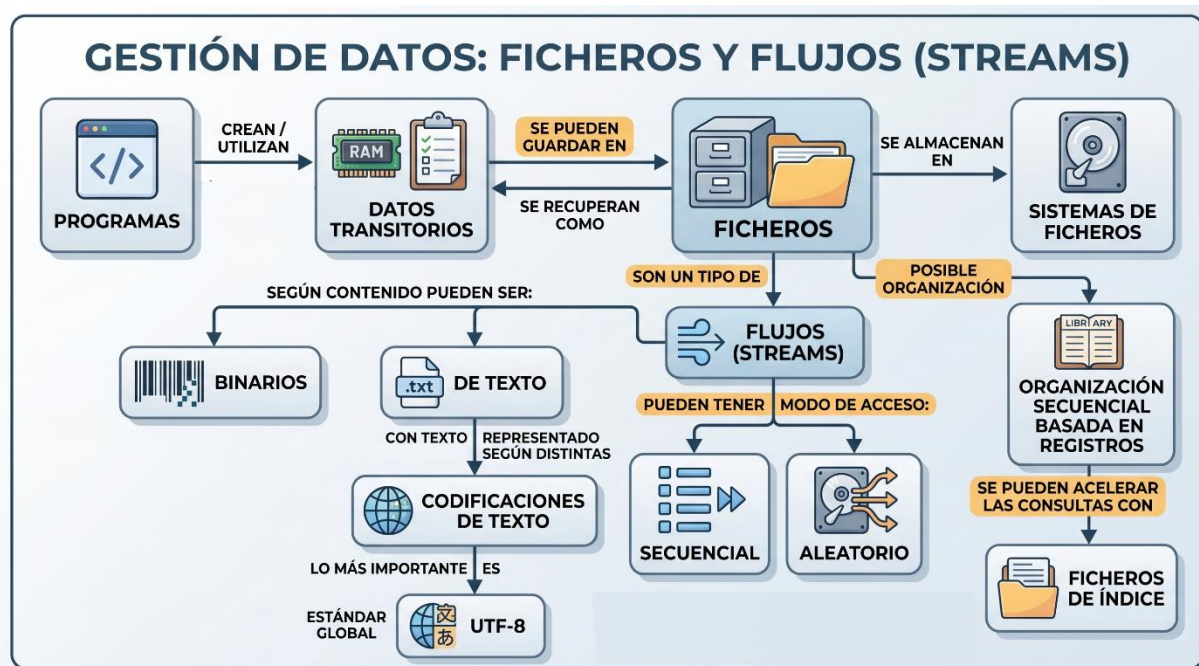


Ambos tipos se usan tanto para **lectura** como para **escritura**. La mayoría de soportes actuales admiten acceso **secuencial y aleatorio**, aunque a veces solo es posible el primero.

Si ampliamos la idea a **flujos de datos (streams)** (de los que un fichero es un caso particular) aparecen más situaciones **solo secuenciales**. Por ejemplo, cuando dos aplicaciones se comunican por **red**, la de origen **envía** y la de destino **recibe** los datos **en orden**.

En muchos lenguajes, **Java** entre ellos, se emplean **las mismas funciones** para **leer/escribir** en ficheros y para **enviar/recibir** por una conexión de red.

1.4 Mapa conceptual



2 Clases para gestión de ficheros y directorios

En Java trabajaremos **principalmente con la clase java.io.File** para representar nombres de **ficheros y directorios**. Con ella podemos **crear, consultar información, renombrar y eliminar**. Algunas facilidades que usaremos para desarrollar los ejemplos de esta Unidad (como indicar el Charset directamente en FileReader/FileWriter, o las utilidades de java.nio.file.Files) requieren **Java 11 o superior**. Si tu entorno usa una versión anterior, algún ejemplo necesitará pequeños ajustes que se irán indicando.

2.1 La clase File

La clase File, perteneciente al paquete java.io, es fundamental para la gestión de ficheros y directorios. Representa nombres de archivos o directorios y permite operaciones como la creación, renombrado, eliminación y consulta de información.

File **no** lee ni escribe contenido; **representa rutas** y ofrece operaciones sobre el **sistema de ficheros**.

<https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/io/File.html>

Categoría	Modificador /tipo	Método(s)	Funcionalidad
Constructor		File(String ruta)	Crea objeto File para la ruta indicada, que puede corresponder a un directorio o a un fichero.
Consulta de propiedades	boolean	canRead() canWrite() canExecute()	Comprueban si el programa tiene diversos tipos de permisos sobre el fichero o directorio, tales como de lectura, escritura y ejecución (si se trata de un fichero). Para un directorio, canExecute() significa que se puede establecer como directorio actual.
	boolean	exists()	Comprueba si el fichero o directorio existe.
	boolean	isDirectory() isFile()	Comprueban si se trata de un directorio o de un fichero.
	long	length()	Devuelve longitud del fichero.
	File	getParent() getParentFile()	Devuelven el directorio padre.
	String	getName()	Devuelve nombre del fichero.
Enumeración	String[]	list()	Devuelve un array con los nombres de los directorios y ficheros dentro del directorio.
	File[]	listFiles()	Devuelve un array con los directorios y ficheros dentro del directorio.
Creación, borrado y renombrado	boolean	createNewFile()	Crea nuevo fichero.
	static File	createTempFile()	Crea nuevo fichero temporal y devuelve objeto de tipo File asociado, para poder trabajar con él.
	boolean	delete()	Borra fichero o directorio.
	boolean	renameTo()	Renombra fichero o directorio.
	boolean	mkdir()	Crea un directorio.
Otras	java.nio.file.Path	toPath()	Devuelve un objeto que permite acceder a información y funcionalidad adicional proporcionada por el paquete java.nio.

El siguiente programa de ejemplo muestra por defecto un listado de los ficheros y directorios que contiene el directorio desde el que se ejecuta el programa. Pero si se le pasa la ruta de un directorio o fichero, muestra información acerca de él y, si se trata de un directorio, muestra los ficheros y directorios que contiene.

```
import java.io.File;
import java.util.Arrays;

public class ListadoDirectorio {

    public static void main(String[] args) {
        // Elegir la ruta: primer argumento o directorio actual "."
        String rutaTexto = (args.length >= 1) ? args[0] : ".";

        // File representa la ruta; crear el objeto NO accede al sistema.
        File fich = new File(rutaTexto);

        // Mostrar la ruta absoluta para evitar confusiones en prácticas.
        System.out.println("Ruta examinada: " + fich.getAbsolutePath());

        // Comprobar existencia antes de intentar leer.
        if (!fich.exists()) {
            System.out.println("No existe el fichero o directorio (" + rutaTexto + ").");
            return;
        }

        // Caso fichero normal
        if (fich.isFile()) {
            System.out.println(rutaTexto + " es un fichero.");
            return;
        }

        // Caso directorio: intentar leer su contenido con control de excepciones.
        if (fich.isDirectory()) {
            System.out.println(rutaTexto + " es un directorio. Contenido:");

            try {
                // listFiles() puede devolver null (p. ej. falta de permisos)
                File[] elementos = fich.listFiles();

                if (elementos == null) {
                    System.out.println("No se puede leer el contenido...");
                    return;
                }

                // Ordenar para que la salida sea determinista en prácticas.
                Arrays.sort(elementos, (a, b) -> a.getName().compareToIgnoreCase(b.getName()));

                // Imprimir cada elemento; marcar directorios con "/".
                for (File e : elementos) {
                    String nombre = e.getName();
                    if (e.isDirectory()) {
                        System.out.println("  " + nombre + "/");
                    } else if (e.isFile()) {
                        System.out.println("  " + nombre);
                    } else {
                        // Otros tipos (p. ej. enlaces) - mostrar como tal.
                        System.out.println("  " + nombre + " (otro)");
                    }
                }
            }
        }
    }
}
```

```
} catch (SecurityException se) {  
    // Acceso denegado al intentar listar el directorio.  
    System.out.println("Permiso denegado al leer el directorio: " + se.getMessage());  
} catch (Exception ex) {  
    // Captura genérica para evitar terminación abrupta; mensaje claro para alumnos.  
    System.out.println("Error al listar el contenido: " + ex.getMessage());  
}  
  
    return;  
}  
  
// Caso raro: existe pero no es fichero ni directorio conocido.  
System.out.println(rutaTexto + " no es ni un fichero ni un directorio reconocido.");  
}
```

Se pueden pasar argumentos desde la línea de comandos a cualquier programa. Cualquier programa Java debe tener un método **main(String args [])** que se invoque en el momento de ejecutar el programa. El parámetro String args [] proporciona los **argumentos de línea de comandos**. Para pasar argumentos de línea de comando a un programa que se esté desarrollando utilizando un IDE (entorno integrado de desarrollo), lo más cómodo suele ser utilizar las opciones que este IDE proporcione para ello dentro de su interfaz de usuario.

3 Operaciones sobre ficheros con Java

Las operaciones básicas son **las mismas** para ficheros de **texto** o **binarios**, y para acceso **secuencial** o **aleatorio**.

El mecanismo de acceso a un fichero está basado en un **puntero** y en una zona de memoria que se suele llamar **buffer**. El puntero siempre apunta a un lugar del fichero, o bien a una posición especial de fin de fichero, que a veces se denomina EOF (del inglés end of file). Esta se puede entender que está situada inmediatamente a continuación del último byte del fichero. Es decir, todo gira en torno a dos ideas:

- **Puntero**: marca la **posición actual** en el fichero (o el **EOF**, *end of file*, justo detrás del último byte).
- **Buffer**: zona de **memoria** usada para mover datos entre el programa y el fichero.

Operaciones comunes en todas las clases de E/S:

1. **Apertura** → al **crear** el objeto de E/S.
2. **Lectura** → read(...): trae datos del fichero al **buffer** y **avanza** el puntero.
3. **Salto** → skip(n): **adelanta** el puntero *n* bytes/caracteres.
4. **Escritura** → write(...): envía datos del buffer al fichero y **avanza** el puntero.
5. **Cierre** → close(): libera recursos.

La diferencia clave es que con **acceso aleatorio** podemos **colocar el puntero** en cualquier posición en cualquier momento. En cambio, con **acceso secuencial**, el puntero solo cambia tras **leer, escribir o saltar**.

3.1 Operaciones de lectura

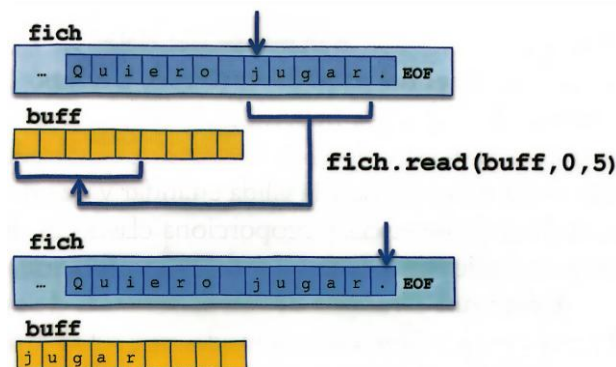
Cuando se lee desde el fichero, hay que indicar el buffer, que recibirá los datos que se lean desde él. Si no se indica el número de bytes o de caracteres para leer, se leerá hasta llenar el buffer. Lógicamente, no se leerá nada si el puntero apunta a la posición EOF (final del fichero).

Lo más destacado de las operaciones de lectura es que:

- Debemos indicar el **buffer** que recibirá los datos.
- Si no se especifica cuántos bytes/caracteres leer, se lee **hasta llenar el buffer**.
- Si el puntero está en **EOF**, **no** se lee nada.
- La lectura funciona igual con **binarios** (se leen **bytes**) y con **texto** (se leen **caracteres**, cuyo tamaño en bytes depende de la **codificación**).
- Para texto por **líneas**, se usa `readLine()` de un **BufferedReader** montado sobre un **FileReader**.

Tras el resultado de la lectura, el **puntero** queda **justo detrás** del último dato leído.

Por ejemplo, para un fichero de texto en el que cada carácter ocupa un byte, la operación de lectura sería:



3.2 Operaciones de escritura

Cuando se escribe en el fichero, hay que indicar el buffer y el número de bytes para escribir. Desde el buffer se transfieren los datos a la posición a la que apunta el puntero. Si el puntero apunta al final del fichero, se añaden los datos al final de este.

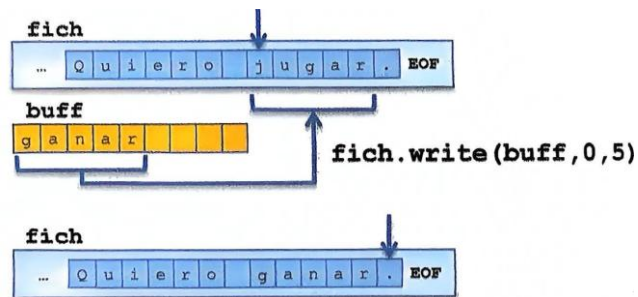
Lo más destacado de las operaciones de escritura es que:

- Indicamos el **buffer** (y, si procede, la **cantidad** a escribir).
- Los datos del buffer se **vuelcan** en la **posición actual** del puntero.
- Si el puntero está al **final del fichero**, la escritura **añade** (append).
- Tras escribir, el puntero queda **justo detrás** del último byte/caracter escrito.

Las **limitaciones más importantes** en las operaciones de escritura son que no existe una forma directa de **insertar** datos en medio del fichero ni de **eliminar** un fragmento arbitrario (salvo

que el reemplazo tenga **exactamente la misma longitud**). Cuando se necesitan estas operaciones, se recurre a **ficheros auxiliares** que permiten reconstruir el contenido deseado.

Por ejemplo:



4 Formas de acceso a ficheros y flujos de datos

En el manejo de ficheros, es fundamental entender los diferentes tipos de acceso que podemos utilizar según el formato y la estructura de los datos almacenados. En Java, las clases del paquete `java.io` nos permiten trabajar tanto con ficheros de acceso secuencial como de acceso aleatorio. Esta distinción es clave para optimizar la eficiencia y la adecuación de la solución a las necesidades de la aplicación.

- **Acceso secuencial:** es la forma básica de trabajar con ficheros. Donde los datos se almacenan y acceden secuencialmente, es decir, para llegar a un dato específico, es necesario leer todos los datos anteriores. En este caso, el formato de los datos está almacenado en texto legible, como secuencia de caracteres. Es posible añadir datos al final de un fichero secuencial, datos al final de un fichero secuencial, aunque esto implica recorrer el fichero hasta el final, lo que puede ser ineficiente en ficheros grandes.
- **Acceso aleatorio:** permite acceder a cualquier parte del fichero directamente, sin necesidad de leer todo el contenido anterior. Esto es posible porque el fichero está organizado en bloques o registros de longitud fija, lo que facilita el salto a cualquier posición dentro del mismo. Más eficiente para leer o modificar datos específicos en ficheros grandes.
- **Acceso binario:** permite acceder directamente a datos en formato de bytes. Es útil para ficheros no legibles como imágenes, música o vídeos. Permite leer o escribir datos en bloques de bytes.

4.1 Acceso secuencial (Flujos de datos – STREAM)

Las operaciones con ficheros secuenciales en Java se realizan utilizando flujos (*streams*, en inglés) mediante **clases del paquete `java.io`**. Un flujo es una **abstracción de alto nivel para cualquier secuencia de datos**. Los ficheros secuenciales son flujos, lo mismo que los datos enviados a través de una conexión de red, o la información contenida en una secuencia de posiciones consecutivas de memoria, o la entrada y la salida estándar y de error de un programa en ejecución. Este paquete saca partido de la herencia, y proporciona **clases con funcionalidad genérica de entrada y salida**, común para cualquier tipo de flujo, y clases con funcionalidad específica para los distintos tipos de flujos.

Existe una jerarquía de clases derivada de cuatro clases, correspondientes a las cuatro combinaciones posibles, considerando por una parte, flujos binarios y de texto y, por otra, flujos de entrada y de salida.

4.1.1 Clases relacionadas con flujos de datos

El tipo de flujo que se ha de crear para realizar operaciones con ficheros secuenciales será distinto según se trate de un **fichero binario o de texto**.

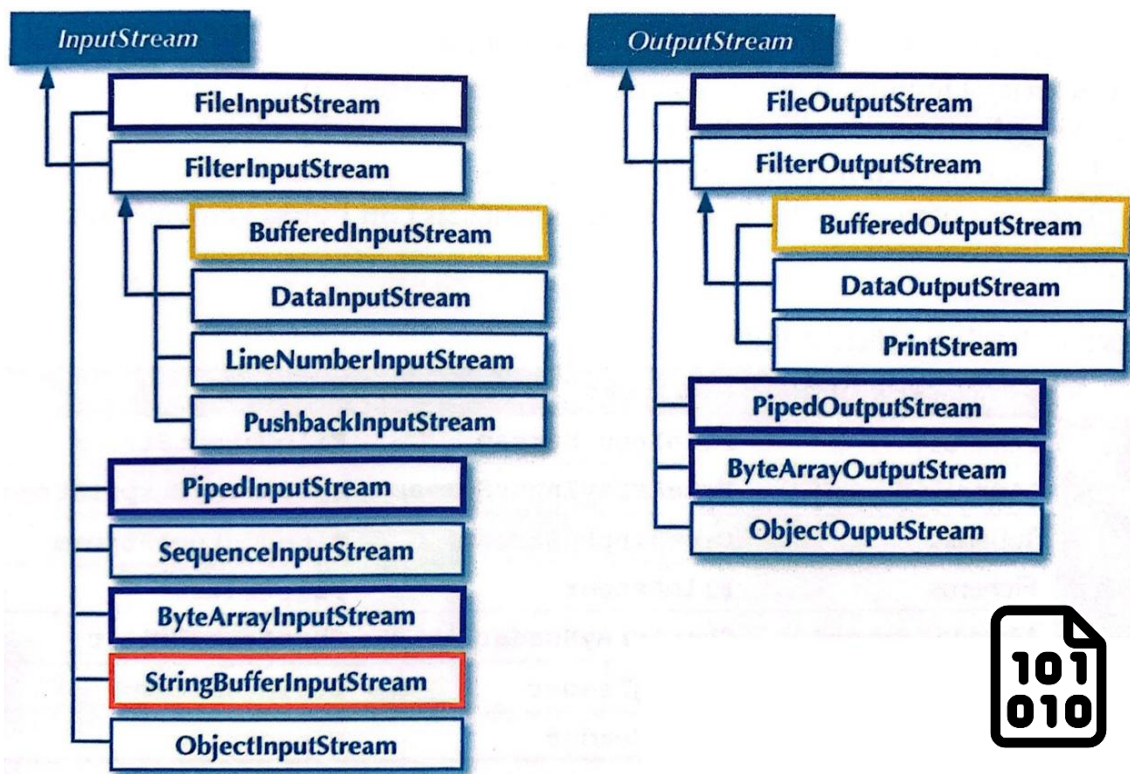
- **Flujos binarios (Byte streams):** Se utilizan para la lectura y escritura de datos binarios. No plantean mayor problema: se leen bytes de un flujo binario hacia una variable de tipo `byte[]`, o se escriben bytes de una variable de tipo `byte[]` a un flujo binario.
- **Flujos de texto (Character streams):** Están orientados a la lectura y escritura de datos de texto. Trabajar con flujos de texto es más complejo debido a las diferentes

codificaciones de texto existentes. Java toma la codificación por defecto de la JVM/SO (en la práctica suele ser UTF-8) pero, en memoria, Java guarda las cadenas en UTF-16 (por tanto, al leer/escribir texto suele haber recodificación entre UTF-8 y UTF-16).

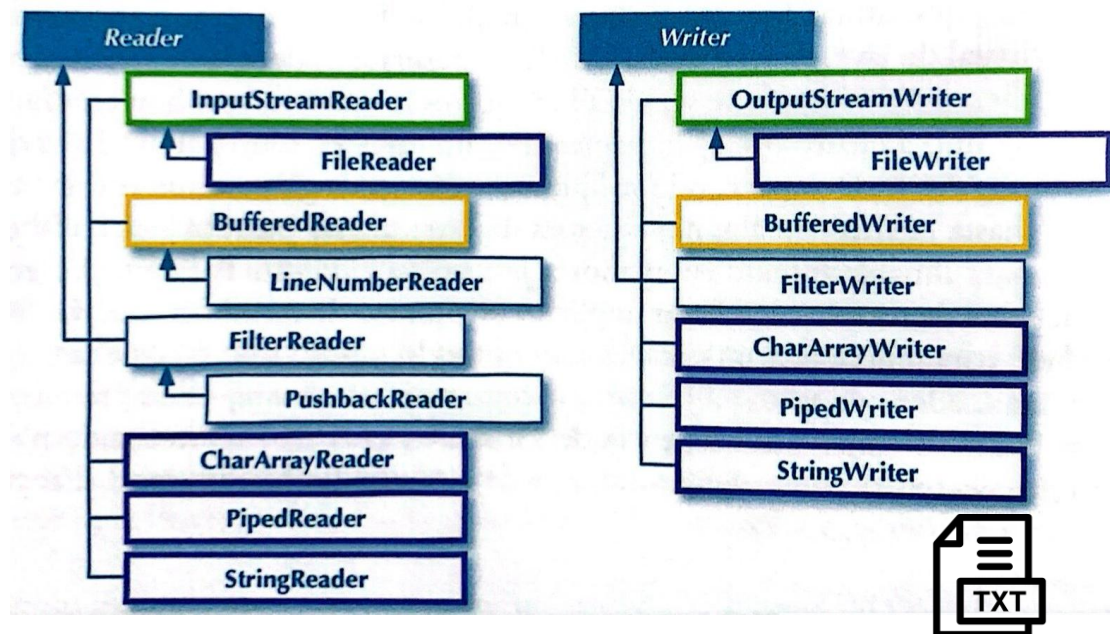
Dentro de cada tipo de flujo, hay diferentes clases que aportan las siguientes funcionalidades:

- **Lectura y escritura:** Funcionalidad básica en flujos asociados a distintos tipos de fuentes de datos.
- **Recodificación de texto:** Java codifica el texto en memoria en UTF-16, en variables de tipo String o char []. La mayoría de ficheros está codificado en UTF-8, y normalmente utiliza esta codificación por defecto para leer y escribir en ellos. Por lo tanto, las operaciones de lectura y escritura de texto en ficheros de texto lleva implícito, generalmente, un proceso de recodificación.
- **Buffering:** Permite aumentar la velocidad de las operaciones de lectura y escritura, y hace posible leer y escribir líneas de texto para los ficheros de texto.

JERARQUÍA DE CLASES PARA FLUJOS BINARIOS



JERARQUÍA DE CLASES PARA FLUJOS DE TEXTO



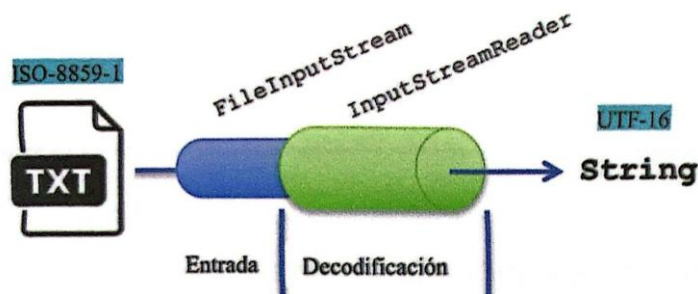
El siguiente cuadro resumen contiene las clases básicas para la entrada y salida de flujos:

	Fuente de datos	Lectura	Escritura
Flujo binario	Ficheros	FileInputStream	FileOutputStream
	Memoria (byte[])	ByteArrayInputStream	ByteArrayOutputStream
	Tuberías	PipedInputStream	PipedOutputStream
Flujo de texto	Ficheros	FileReader	FileWriter
	Memoria (char[])	CharArrayReader	CharArrayWriter
	Memoria (String)	StringReader	StringWriter
	Tuberías	PipedReader	PipedWriter

4.1.2 Clases para recodificación

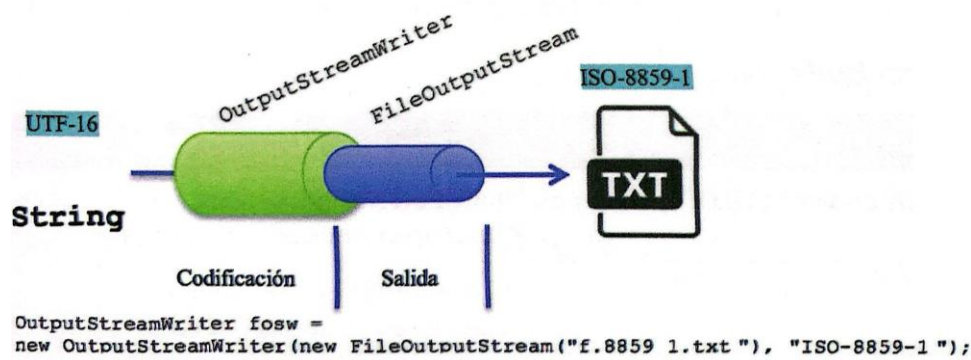
Las clases `InputStreamReader` y `OutputStreamWriter` sirven de enlace entre ambas jerarquías: la de flujos binarios y la de flujos de texto. Se suelen utilizar para convertir flujos binarios en flujos de texto y viceversa, y es necesario especificar una codificación a su constructor.

Conceptualmente, las siguientes imágenes muestran algunos ejemplos:



```
InputStreamReader fisr =
new InputStreamReader(new FileInputStream("f.8859_1.txt"), "ISO-8859-1");
```

Lectura desde ficheros de texto con codificación distinta a la codificación por defecto



Escritura a ficheros de texto con codificación distinta a la codificación por defecto

4.1.3 Clases para buffering

Las clases existentes para buffering aceleran las operaciones de lectura o escritura, utilizando una zona de intercambio en memoria llamada buffer. Permiten leer y escribir líneas de texto en ficheros de texto.

Es recomendable utilizar las clases que proporcionan buffering. El rendimiento aumenta para ficheros grandes y cuando se realizan lecturas o escrituras consecutivas en posiciones contiguas, que es lo habitual.

Las clases principales que utilizan buffering son:

	LECTURA	ESCRITURA
Flujo binario	BufferedInputStream	BufferedOutputStream
Flujo de texto	BufferedReader	BufferedWriter

Los métodos más destacados para lectura y escritura de líneas en clases para buffering con **ficheros de texto** son:

Clase	Método	Funcionalidad
BufferedReader	String readLine()	Lee hasta el final de la línea actual.
BufferedWriter	void newLine()	Escribe un separador de líneas. El separador de líneas puede depender del sistema operativo, y suele ser distinto en Linux y en Windows. El método readLine() de BufferedReader tiene en cuenta estas particularidades.

4.1.4 Operaciones de lectura para flujos de entrada

Son operaciones similares para todas las clases que implementan flujos (streams) de entrada en Java (independientemente de si son binarios o de texto):

- Las funciones **read** de clases que heredan de **InputStream** leen bytes.
- Las funciones **read** que heredan de **Reader** leen caracteres.

Función	Descripción
<code>read()</code>	Lee un byte o un carácter hasta llenar el buffer o el número de bytes o caracteres indicados, que se copiarán en la posición indicada (offset) dentro del buffer. Devuelven el número de bytes o caracteres leídos, o -1 si no se pudo leer nada porque el puntero estaba al final del fichero.
<code>skip()</code>	Saltan el número indicado de bytes o caracteres , aunque podrían ser menos si se alcanza el final del fichero. En cualquier caso, devuelven el número de bytes o de caracteres que se han saltado.
<code>readLine()</code>	Con ficheros de texto, para leer líneas se puede utilizar el método <code>readLine()</code> de un <code>BufferedReader</code> construido sobre un <code>FileReader</code>.

El siguiente cuadro muestra los métodos de lectura de las clases para la gestión de flujos de entrada (en función de si se trata de datos binarios o de texto):

InputStream	Reader
<code>int read()</code>	<code>int read()</code>
<code>int read(byte[] buffer)</code>	<code>int read(char[] buffer)</code>
<code>int read(byte[] buffer, int offset, int longitud)</code>	<code>int read(char[] buffer, int offset, int longitud)</code>
	<code>int read(CharBuffer buffer)</code>
<code>long skip(long n)</code>	<code>long skip(long n)</code>
	BufferedReader
	<code>String readLine()</code>

4.1.5 Operaciones de escritura para flujos de salida

Son similares para todas las clases que implementan flujos (streams) de salida en Java.

Las funciones **write**:

- Clases que heredan de `OutputStream` escriben bytes.
- Clases que heredan de `Writer` escriben caracteres.

Según la variante, escriben un byte o un carácter, o todos los contenidos del buffer, o el número de bytes o caracteres indicados a partir de la posición indicada. Si se alcanza el fin del fichero, siguen escribiendo. Las que heredan de `Writer` tienen métodos para escribir los contenidos de un `String`, y **métodos `append`** para añadir al final del fichero.

El siguiente cuadro muestra los métodos de escritura de las clases para la gestión de flujos de salida (en función de si se trata de datos binarios o de texto):

OutputStream	Writer
<pre>void write(int b) void write(byte[] buffer) void write(byte[] buffer, int offset, int longitud)</pre>	<pre>void write(int c) void write(char[] buffer) void write(char[] buffer, int offset, int longitud) void write(String str) void write(String str, int offset, int longitud) Writer append(char c) Writer append(CharSequence csq) Writer append(CharSequence csq, int offset, int longitud)</pre>
BufferedWriter	
<pre>void newLine()</pre>	

Ejemplos prácticos de lectura y escritura secuencial

Hasta ahora hemos visto **qué clases** existen y **qué métodos** ofrecen. Vamos a juntar esas piezas en programas completos y ejecutables. Todos los ejemplos siguen tres buenas prácticas que conviene adoptar desde el principio:

- **try-with-resources:** el recurso (flujo) se declara entre paréntesis tras el try y Java lo **cierra automáticamente** al salir del bloque, incluso si se produce una excepción. Evita fugas de recursos y hace innecesario el close() manual.
- **Codificación explícita:** indicamos siempre StandardCharsets.UTF_8. Así el programa se comporta igual en cualquier máquina, sin depender de la codificación por defecto del sistema (recuerda lo visto sobre recodificación UTF-8 ↔ UTF-16).
- **Buffering:** envolvemos los flujos en BufferedReader/BufferedWriter (texto) para ganar rendimiento y poder trabajar por líneas.

Ejemplo 1.- Escritura de un fichero de texto, línea a línea

```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;
import java.nio.charset.StandardCharsets;

public class EscrituraTexto {
    public static void main(String[] args) {
        // El flujo se declara dentro del try: se cerrará solo al terminar.
        // El segundo parámetro 'false' significa "sobrescribir"; 'true' sería "añadir al final"
        (append).
        try (BufferedWriter bw = new BufferedWriter(
            new FileWriter("agenda.txt", StandardCharsets.UTF_8, false))) {

            bw.write("Ana Torres;600111222");
            bw.newLine(); // salto de línea independiente del sistema operativo
            bw.write("Luis Gómez;600333444");
            bw.newLine();

            System.out.println("Fichero 'agenda.txt' escrito correctamente.");

        } catch (IOException e) {
            System.err.println("Error al escribir el fichero: " + e.getMessage());
        }
    }
}
```

Ejemplo 2.- Lectura de un fichero de texto, línea a línea

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
import java.nio.charset.StandardCharsets;

public class LecturaTexto {
    public static void main(String[] args) {
        try (BufferedReader br = new BufferedReader(
            new FileReader("agenda.txt", StandardCharsets.UTF_8))) {

            String linea;
            // readLine() devuelve la siguiente línea, o null cuando se llega al final (EOF).
            while ((linea = br.readLine()) != null) {
                String[] campos = linea.split(";"); // "nombre;teléfono"
                System.out.println("Nombre: " + campos[0] + " | Tel: " + campos[1]);
            }

        } catch (IOException e) {
            System.err.println("Error al leer el fichero: " + e.getMessage());
        }
    }
}
```

Ejemplo 3.- Escritura y lectura de datos binarios

```
import java.io.*;

public class DatosBinarios {
    public static void main(String[] args) {

        // --- Escritura binaria ---
        try (DataOutputStream dos = new DataOutputStream(
            new BufferedOutputStream(new FileOutputStream("producto.dat")))) {

            dos.writeUTF("Teclado mecánico"); // cadena
            dos.writeInt(45); // unidades en stock
            dos.writeDouble(79.95); // precio
            dos.writeBoolean(true); // disponible

        } catch (IOException e) {
            System.err.println("Error al escribir datos binarios: " + e.getMessage());
        }

        // --- Lectura binaria (mismo orden que la escritura) ---
        try (DataInputStream dis = new DataInputStream(
            new BufferedInputStream(new FileInputStream("producto.dat")))) {

            String nombre = dis.readUTF();
            int stock = dis.readInt();
            double precio = dis.readDouble();
            boolean disp = dis.readBoolean();

            System.out.printf("%s | stock: %d | precio: %.2f € | disponible: %b%n",
                nombre, stock, precio, disp);

        } catch (IOException e) {
            System.err.println("Error al leer datos binarios: " + e.getMessage());
        }
    }
}
```

Ejemplo 4.- Copia de fichero byte a byte (con buffer)

```
import java.io.*;

public class CopiaFichero {
    public static void main(String[] args) {
        String origen = "entrada.jpg";
        String destino = "copia.jpg";

        try (BufferedInputStream in = new BufferedInputStream(new FileInputStream(origen));
            BufferedOutputStream out = new BufferedOutputStream(new FileOutputStream(destino))) {

            byte[] buffer = new byte[8192]; // leemos en bloques de 8 KB, no byte a byte
            int leidos;
            // read() devuelve el número de bytes leídos, o -1 al llegar al final del fichero.
            while ((leidos = in.read(buffer)) != -1) {
                out.write(buffer, 0, leidos); // escribimos solo los bytes realmente leídos
            }

            System.out.println("Copia completada: " + origen + " → " + destino);

        } catch (IOException e) {
            System.err.println("Error durante la copia: " + e.getMessage());
        }
    }
}
```

4.2 Acceso aleatorio

Para el acceso aleatorio a ficheros se utiliza la clase **RandomAccessFile**. Las novedades con respecto a lo ya visto hasta ahora son:

1. En todo momento el cursor se puede situar en cualquier posición mediante la función **seek()**. En eso consiste el acceso aleatorio.
2. Sobre el fichero se pueden realizar operaciones tanto de lectura como de escritura.

A pesar de ello, el acceso aleatorio a los ficheros tiene una serie de limitaciones. Limitaciones que no vienen del lenguaje Java (ni del paquete java.io), sino de los sistemas de ficheros habituales:

1. No es posible eliminar o insertar bloques de bytes o de caracteres en mitad de un fichero, ni reemplazar un bloque de un fichero por otro, a no ser que tengan exactamente el mismo tamaño en bytes.
2. Es necesario el uso de ficheros temporales auxiliares.

La clase **RandomAccessFile** posee dos constructores:

- **RandomAccessFile(File file, String mode)**
- **RandomAccessFile(String name, String mode)**

Se puede pasar un objeto **File** como primer parámetro, o un **String**. Y el modo es: "r" si se abre en modo lectura o "rw" si se abre en modo lectura y escritura.

4.2.1 Operaciones con ficheros

Los principales métodos para ficheros de acceso aleatorio son:

- [seek\(long posicion\)](#): Permite mover el puntero de lectura/escritura a una posición específica de un objeto RandomAccessFile.
- [getFilePointer\(\)](#): Devuelve la posición actual del puntero dentro del objeto RandomAccessFile.
- [skipBytes\(int desplazamiento\)](#): Permite saltar un número específico de bytes desde la posición actual del puntero.
- [length\(\)](#): Devuelve la longitud total del archivo en bytes.

Método	Funcionalidad
RandomAccessFile(File file, String mode) RandomAccessFile(String name, String mode)	Constructor. Abre el fichero en el modo indicado, si se dispone de permisos suficientes: - r: modo de solo lectura. - rw: modo de lectura y escritura. - rwd, rws: Como rw pero con escritura síncrona. Esto significa que todas las operaciones de escritura (de datos con rwd y de datos y metadatos con rws) deben haberse completado cuando termina la función. Esto puede hacer que la llamada a la función tarde más, pero asegura que no se pierde información crítica ante una caída del sistema.
void close()	Cierra el fichero. Es conveniente hacerlo siempre al final.
void seek(long pos)	Posiciona el puntero en la posición indicada.
int skipBytes(int n)	Intenta avanzar el puntero el número de bytes indicado. Se devuelve el número de bytes que se ha avanzado. Podría ser menor que el solicitado, si se alcanza el fin del fichero.
int read() int read(byte[] buffer) int read(byte[] buffer, int offset, int longitud)	Lee del fichero. Según la variante, un byte o hasta llenar el buffer, o el número de bytes indicados, que se copiarán en la posición indicada (offset) del buffer. Devuelve el número de bytes leídos, o -1 si no se pudo leer nada porque el puntero estaba al final del fichero.
void readFully(byte[] buffer) readFully(byte[] buffer, int offset, int longitud)	Como <code>int read(byte[] b)</code> , pero si no se puede leer hasta llenar el buffer, o el número de bytes indicado, porque se llega al final del fichero, se lanza la excepción <code>IOException</code> . Útil cuando se sabe que se podrá leer hasta llenar el buffer o el número de bytes indicados. En cualquier caso, la eventualidad de que no se pueda completar la lectura se puede gestionar capturando la excepción.
String readLine()	Lee hasta el final de la línea de texto actual.
void write(int b) void write(byte[] buffer) void write(byte[] buffer, int offset, int longitud)	Escribe en el fichero. Según la variante, un byte, o todos los contenidos del buffer, o el número de bytes indicados a partir de la posición indicada (offset). Si alcanza el fin del fichero, siguen escribiendo.

Ejemplo práctico: registros de longitud fija

La clave del acceso aleatorio está en poder **calcular** dónde empieza cada dato sin haber leído lo anterior. Esto solo es posible si todos los registros **ocupan exactamente lo mismo**: son registros de **longitud fija**. Si cada registro mide T bytes, el registro número i (empezando en 0) empieza en la posición:

$$\text{posición} = i * T$$

Basta entonces con seek(i * T) para saltar directamente a él. Ese es el mecanismo que usan muchas bases de datos sencillas basadas en ficheros.

En la práctica, los campos de texto tienen longitudes variables, y necesitamos que **no** las tengan. La técnica habitual es reservar un número fijo de caracteres por campo y rellenar (o recortar) hasta esa longitud. Veámoslo con una agenda de contactos donde cada registro ocupa siempre lo mismo:

- **nombre** → 20 caracteres (40 bytes, porque writeChars usa 2 bytes por carácter)
- **teléfono** → un int (4 bytes)

Tamaño de registro: 40 + 4 = 44 bytes.

Ejemplo 5.- Agenda con acceso aleatorio

```
import java.io.IOException;
import java.io.RandomAccessFile;

public class AgendaAleatoria {

    // Cada registro ocupa SIEMPRE lo mismo, para poder calcular su posición.
    static final int LONG_NOMBRE = 20;           // caracteres reservados al nombre
    static final int BYTES_NOMBRE = LONG_NOMBRE * 2; // writeChars/readChar usan 2 bytes por
carácter
    static final int BYTES_TELEFONO = 4;         // un int
    static final int TAM_REGISTRO = BYTES_NOMBRE + BYTES_TELEFONO; // 44 bytes

    public static void main(String[] args) {

        // "rw": abrimos en lectura y escritura.
        try (RandomAccessFile raf = new RandomAccessFile("agenda.dat", "rw")) {

            // 1) Escribimos tres registros consecutivos.
            escribirRegistro(raf, 0, "Ana Torres", 600111222);
            escribirRegistro(raf, 1, "Luis Gómez", 600333444);
            escribirRegistro(raf, 2, "Marta Ruiz", 600555666);

            // 2) Leemos DIRECTAMENTE el registro 2, sin recorrer los anteriores.
            System.out.println("Registro 2 -> " + leerRegistro(raf, 2));
        }
    }
}
```

```
// 3) Modificamos SOLO el teléfono del registro 1 (acceso directo + reescritura).
raf.seek(1 * TAM_REGISTRO + BYTES_NOMBRE); //saltamos al campo teléfono del registro 1
raf.writeInt(699999999);

// 4) Comprobamos el cambio.
System.out.println("Registro 1 -> " + leerRegistro(raf, 1));

} catch (IOException e) {
    System.err.println("Error de acceso al fichero: " + e.getMessage());
}

}

/** Escribe un registro en la posición 'indice', rellenando el nombre a longitud fija. */
static void escribirRegistro(RandomAccessFile raf, int indice, String nombre, int telefono)
    throws IOException {
    raf.seek((long) indice * TAM_REGISTRO); // nos situamos al inicio del registro
    raf.writeChars(ajustar(nombre, LONG_NOMBRE));
    raf.writeInt(telefono);
}

/** Lee el registro situado en la posición 'indice'. */
static String leerRegistro(RandomAccessFile raf, int indice) throws IOException {
    raf.seek((long) indice * TAM_REGISTRO);
    StringBuilder nombre = new StringBuilder();
    for (int i = 0; i < LONG_NOMBRE; i++) {
        nombre.append(raf.readChar()); // leemos exactamente 20 caracteres
    }
    int telefono = raf.readInt();
    return nombre.toString().trim() + " - " + telefono;
}

/** Recorta o rellena con espacios para que el texto ocupe SIEMPRE 'longitud' caracteres. */
static String ajustar(String texto, int longitud) {
    if (texto.length() > longitud) {
        return texto.substring(0, longitud); // recortamos si sobra
    }
    return String.format("%-" + longitud + "s", texto); // rellenamos con espacios si falta
}
}
```

4.3 Aplicaciones reales e importancia de la gestión de ficheros



A. Archivos de registro (logs).

En muchas aplicaciones, los archivos de registro son utilizados para almacenar datos de eventos de forma secuencial. Cada nuevo registro se añade al final del fichero.



B. Bases de datos (sistemas sencillos basados en archivos):

Algunas aplicaciones utilizan ficheros binarios para implementar una base de datos simple donde los registros están organizados en bloques de tamaño fijo y el acceso aleatorio facilita las operaciones de consulta o modificación. Además, los flujos son cruciales para la gestión de datos en sistemas de bases de datos simples que almacenan información en archivos. Permiten leer y escribir datos de forma eficiente, facilitando la persistencia de la información.



C. Archivos y aplicaciones multimedia

Los **ficheros binarios** son comúnmente usados para almacenar datos multimedia como imágenes, vídeos y música. Estos ficheros necesitan ser accedidos aleatoriamente para poder modificar, extraer o reproducir secciones específicas del contenido. En **reproductores multimedia**, los flujos con búfer son esenciales para minimizar la latencia. El búfer permite una reproducción fluida, evitando interrupciones causadas por la espera de datos desde el disco o la red.



5 Trabajo con ficheros XML

En los apartados anteriores has trabajado con ficheros de texto y binarios en los que la estructura de los datos la decidía el propio programa: una línea por registro, campos separados por punto y coma, registros de longitud fija... Esa libertad tiene un precio: solo el programa que creó el fichero sabe interpretarlo. Si otra aplicación, escrita en otro lenguaje o por otra empresa, necesita leer esos datos, alguien tiene que explicarle antes cómo están organizados.

XML resuelve este problema. En lugar de que cada programa invente su propio formato, todos siguen las mismas reglas para describir los datos, y la estructura queda recogida dentro del propio fichero. En este apartado verás qué es XML y los conceptos que necesitas para trabajar con él desde Java. En los siguientes aprenderás a leerlo (5.2), a convertirlo directamente en objetos (5.3) y a transformarlo en otros formatos (5.4).

5.1 Conceptos clave

¿Qué es XML?

XML (*eXtensible Markup Language*) es un lenguaje de marcas creado para estructurar, almacenar e intercambiar datos. Más exactamente, es un **metalenguaje**: no define un vocabulario concreto de etiquetas, sino las reglas para que cada uno cree el suyo. A diferencia de HTML, donde las etiquetas están fijadas de antemano (`<p>`, `<h1>`, `<table>`), en XML las etiquetas las define el programador según los datos que quiera representar.

Tres características definen un documento XML:

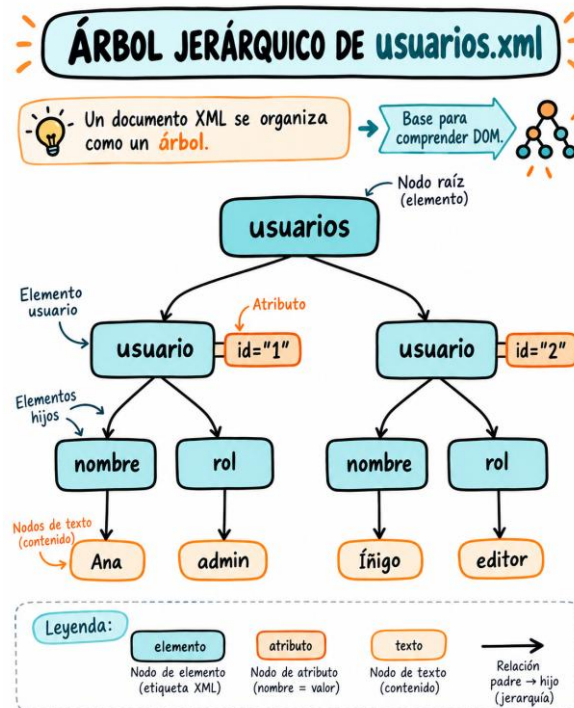
- **Lenguaje de marcas.** Los datos se encierran entre etiquetas que indican qué significa cada uno. `<nombre>Ana</nombre>` no solo guarda el texto «Ana»: dice, además, que es un nombre.
- **Estructura jerárquica.** Todo documento tiene un único elemento raíz que contiene al resto, y los elementos se anidan unos dentro de otros formando un árbol.
- **Texto plano.** Un fichero XML se puede abrir con cualquier editor de texto. Lo leen tanto las personas como los programas.

Observa el siguiente documento, `usuarios.xml`. Es el ejemplo que vamos a utilizar a lo largo de todo el apartado 5:

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <usuarios>
3   <usuario id="1">
4     <nombre>Ana</nombre>
5     <rol>admin</rol>
6   </usuario>
7   <usuario id="2">
8     <nombre>Íñigo</nombre>
9     <rol>editor</rol>
10  </usuario>
11 </usuarios>
```

En él puedes distinguir todas las piezas de un documento XML:

- **Declaración.** La primera línea indica la versión de XML y la codificación de caracteres. Gracias a `encoding="UTF-8"`, el nombre «Íñigo» se leerá correctamente en cualquier sistema.
- **Elemento raíz.** `<usuarios>` contiene todo el documento. Solo puede haber uno.
- **Elementos.** Cada pareja de etiquetas de apertura y cierre, como `<usuario> ... </usuario>` o `<nombre> ... </nombre>`.
- **Atributos.** Datos que acompañan a la etiqueta de apertura, como `id="1"`. Se suelen usar para identificadores o para propiedades breves.
- **Contenido.** El texto que hay entre una etiqueta de apertura y su cierre, como Ana o admin.



¿Por qué se sigue usando XML?

Hoy existen formatos más ligeros, como JSON, y aun así XML sigue presente en muchísimos sistemas. Hay tres razones principales:

- **Interoperabilidad.** Permite que se comuniquen plataformas, lenguajes y redes distintas. Un programa en Java puede generar un XML que después lea una aplicación escrita en Python o en C# sin necesidad de ningún acuerdo adicional.
- **Estandarización.** Sus reglas las define el W3C (*World Wide Web Consortium*), de modo que cualquier herramienta que siga el estándar entiende cualquier documento XML.
- **Accesibilidad.** Tanto el formato como la forma de acceder a él son estándar, y además un documento se puede validar contra un esquema para garantizar que sus datos tienen la estructura esperada.

De hecho, lo usas más de lo que crees. El fichero `pom.xml` con el que Maven gestiona las dependencias de tus proyectos es XML; las pantallas de las aplicaciones Android se diseñan en ficheros XML; un documento de Word (`.docx`) es en realidad un archivo comprimido que

contiene varios ficheros XML; y la factura electrónica en España, en su formato Facturae, también es XML.

Documento bien formado y documento válido

Para que un programa pueda procesar un documento XML, este tiene que cumplir unas reglas. Hay dos niveles de exigencia que conviene no confundir.

Un documento está **bien formado** cuando respeta la sintaxis de XML:

- Tiene un único elemento raíz.
- Toda etiqueta que se abre se cierra, y en el orden correcto: si `<usuario>` contiene a `<nombre>`, el cierre `</nombre>` tiene que aparecer antes que `</usuario>`.
- Distingue mayúsculas de minúsculas: `<Nombre>` y `<nombre>` son etiquetas distintas.
- Los valores de los atributos van entre comillas.

El siguiente fragmento, por ejemplo, no está bien formado: las etiquetas se cierran en orden incorrecto y el valor del atributo no lleva comillas.

```
1 <usuario id=1>
2   <nombre>Ana</usuario>
3 </nombre>
```

Un documento es **válido** cuando, además de estar bien formado, cumple las reglas de un esquema: qué elementos puede contener, en qué orden, cuántas veces pueden aparecer y de qué tipo son sus datos. El esquema funciona como la plantilla que describe cómo deben ser todos los documentos de un determinado tipo.

Bien formado no es lo mismo que válido: lo primero es cuestión de sintaxis; lo segundo exige un esquema.

De aquí se deduce una regla sencilla: todo documento válido está bien formado, pero un documento bien formado no tiene por qué ser válido. El documento `usuarios.xml` que has visto antes está bien formado; si además disponemos de un esquema que describa cómo debe ser un documento de usuarios, podremos comprobar también si es válido.

Parsing, esquema y binding

Cuando un programa trabaja con XML aparecen tres conceptos que se van a repetir durante toda esta parte de la unidad.

Parsing (análisis léxico y sintáctico). Es el proceso de leer un documento XML, comprobar que está bien formado (y, si hay un esquema, que es válido) y entregar después su contenido a la aplicación que lo necesita. El programa que se encarga de hacerlo se llama **parser** o analizador. Sin él, tu programa tendría que leer el fichero carácter a carácter y reconocer por su cuenta cada etiqueta, algo muy laborioso y propenso a errores. En Java no tienes que programarlo: el JDK ya incluye varios, que estudiarás en el apartado 5.2.

Schema (esquema). Especifica los elementos permitidos en un documento XML, su orden y sus relaciones, y es lo que permite decir si un documento es válido. El formato más utilizado es **XSD** (*XML Schema Definition*), que es a su vez un documento XML. Ya trabajaste con esquemas en el módulo de Lenguajes de marcas de primer curso. En esta unidad no vas a

escribirlos, pero sí debes saber reconocer uno. Este es un esquema para el documento de usuarios:

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
3      <xs:element name="usuarios">
4          <xs:complexType>
5              <xs:sequence>
6                  <xs:element name="usuario" maxOccurs="unbounded">
7                      <xs:complexType>
8                          <xs:sequence>
9                              <xs:element name="nombre" type="xs:string"/>
10                             <xs:element name="rol" type="xs:string"/>
11                         </xs:sequence>
12                         <xs:attribute name="id" type="xs:int" use="required"/>
13                     </xs:complexType>
14                 </xs:element>
15             </xs:sequence>
16         </xs:complexType>
17     </xs:element>
18 </xs:schema>

```

Fíjate en que el esquema no contiene datos, sino reglas: `usuarios` contiene uno o más elementos `usuario` (`maxOccurs="unbounded"`); cada `usuario` tiene un `nombre` y un `rol`, en ese orden; y lleva un atributo `id` numérico obligatorio (`use="required"`). Un documento de usuarios al que le faltara el `rol` seguiría estando bien formado, pero ya no sería válido.

Binding (vinculación). Es el proceso de enlazar un documento XML con objetos de Java, de manera que cada elemento del documento se corresponda con un objeto o con uno de sus campos. Con un parser, tu programa recorre el documento etiqueta a etiqueta; con binding, trabaja directamente con objetos (por ejemplo, con objetos de una clase `Usuario`) y es la herramienta la que se encarga de la conversión en los dos sentidos. En Java, esta tarea la realiza **JAXB**, que estudiarás en el apartado 5.3.

Nota: algunas herramientas son capaces de generar automáticamente las clases Java a partir de un esquema XSD. En esta unidad no lo harás así: escribirás tú las clases y les indicarás con anotaciones cómo se corresponden con el XML, como verás en el apartado 5.3.

La siguiente tabla resume los tres conceptos:

Concepto	Qué hace	Dónde lo trabajarás
Parsing	Lee el documento y comprueba que es correcto	5.2. Parser o analizador XML
Esquema	Define las reglas que debe cumplir el documento	Solo lo reconocerás; no lo escribirás
Binding	Convierte el documento en objetos Java y al revés	5.3. Vinculación o binding (JAXB)

5.2 Parser o analizador XML

En el apartado anterior has visto que un **parser** es el programa que lee un documento XML, comprueba que es correcto y entrega su contenido a tu aplicación. En este apartado vas a ver

qué tipos de parser existen, cómo funcionan los dos más utilizados en Java (SAX y DOM) y cómo leer con DOM el documento `usuarios.xml`.

Tipos de parsers

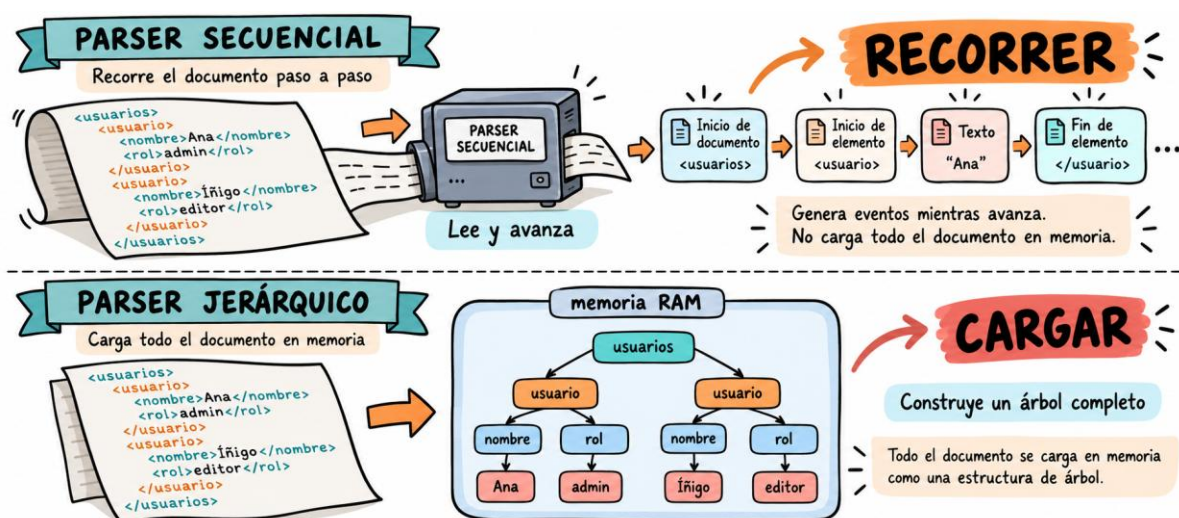
Todos los parsers leen el documento y comprueban su sintaxis, pero no todos entregan el resultado de la misma forma. Se distinguen dos familias:

- **Parsers secuenciales.** Analizan el documento de principio a fin, una sola vez. A medida que encuentran las etiquetas de apertura y de cierre, van extrayendo el contenido y avisando a tu programa. Son muy rápidos y apenas consumen memoria, pero no guardan nada: para encontrar un dato concreto tienen que recorrer el documento hasta llegar a él.
- **Parsers jerárquicos.** Cargan todo el documento en memoria RAM y lo organizan siguiendo su jerarquía, es decir, como un árbol. Una vez cargado, puedes consultarlo tantas veces como quieras y en cualquier orden, e incluso modificarlo. Por eso son los más utilizados por las aplicaciones que necesitan trabajar de forma continuada con los datos.

	Secuencial	Jerárquico
Cómo lee	De principio a fin, lanzando eventos	Carga el documento entero en memoria
Memoria	Muy poca	Proporcional al tamaño del fichero
Modificar	No lo permite	Permite añadir, cambiar y borrar nodos
En Java	SAX, StAX	DOM

¿Cómo elegir? La regla práctica es sencilla. Si el fichero es grande y solo hay que leerlo, conviene un parser secuencial. Si hay que consultarlo varias veces o modificarlo, conviene uno jerárquico.

Nota: DOM no es una tecnología exclusiva de Java. Es una especificación del W3C que implementan muchos lenguajes; por ejemplo, el navegador usa DOM para representar las páginas web que manipulas con JavaScript.



Parser SAX: lectura por eventos

SAX (*Simple API for XML*) es el parser secuencial más conocido de Java y viene incluido en el JDK. Su funcionamiento se basa en **eventos**: recorre el documento una sola vez, de principio a fin, y cada vez que encuentra algo significativo (el inicio del documento, una etiqueta de apertura, un texto, una etiqueta de cierre...) dispara un evento. Tu trabajo como programador es escribir los métodos que responden a esos eventos.

Los cinco eventos principales se corresponden con cinco métodos de la clase `DefaultHandler`, que tú sobrescribes en una clase propia:

- `startDocument()` y `endDocument()`: se ejecutan al principio y al final del documento.
- `startElement()` y `endElement()`: se ejecutan al encontrar una etiqueta de apertura y una de cierre. Reciben el nombre de la etiqueta y, en el caso de la apertura, sus atributos.
- `characters()`: se ejecuta con el texto que hay dentro de una etiqueta.

El siguiente programa lee `usuarios.xml` con SAX y muestra el identificador y el nombre de cada usuario:

```
import java.io.File;
import javax.xml.parsers.SAXParser;
import javax.xml.parsers.SAXParserFactory;
import org.xml.sax.Attributes;
import org.xml.sax.helpers.DefaultHandler;

public class LectorSAX {
    // throws Exception para centrar el ejemplo en SAX; la gestión
    // correcta de las excepciones la verás en el apartado 6
    public static void main(String[] args) throws Exception {
        SAXParserFactory fabrica = SAXParserFactory.newInstance();
        SAXParser parser = fabrica.newSAXParser();
        parser.parse(new File("usuarios.xml"), new ManejadorUsuarios());
    }
}

class ManejadorUsuarios extends DefaultHandler {
    private final StringBuilder texto = new StringBuilder();

    @Override
    public void startElement(String uri, String localName, String qName,
        Attributes atributos) {
        texto.setLength(0); // empieza una etiqueta: vaciamos
        if (qName.equals("usuario")) {
            System.out.println("Usuario con id " + atributos.getValue("id"));
        }
    }

    @Override
    public void characters(char[] ch, int inicio, int longitud) {
        texto.append(ch, inicio, longitud); // puede llegar en varios trozos
    }

    @Override
    public void endElement(String uri, String localName, String qName) {
        if (qName.equals("nombre")) {

```

```
System.out.println(" Nombre: " + texto.toString().trim());
}
}
```

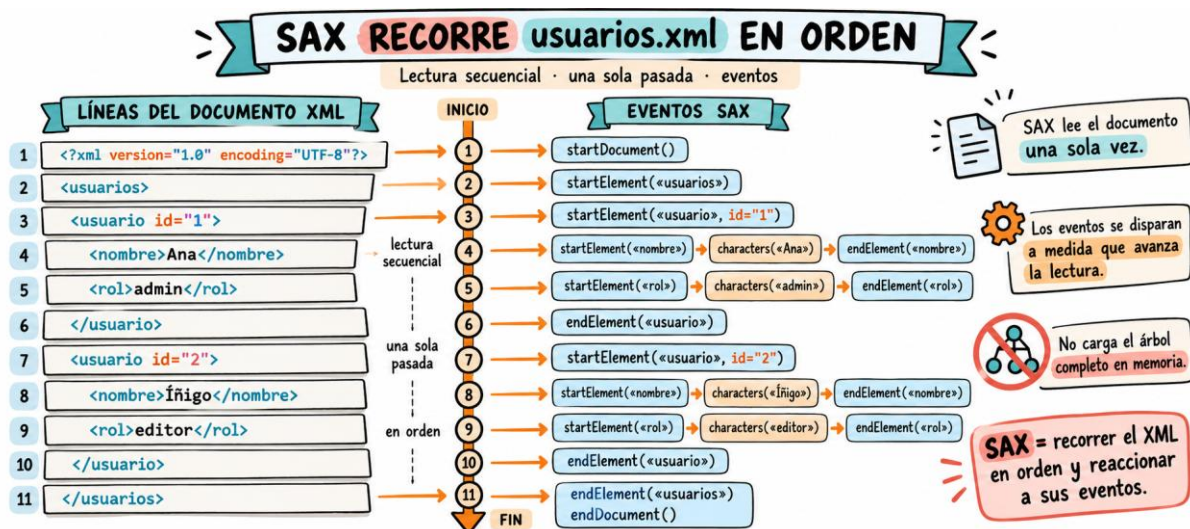
Al ejecutarlo, obtendrás esta salida:

```
1 Usuario con id 1
2 Nombre: Ana
3 Usuario con id 2
4 Nombre: Íñigo
```

Fíjate en dos detalles. El primero: el método `characters()` puede recibir el texto de una etiqueta en varios trozos, así que lo vamos acumulando en un `StringBuilder` y lo usamos al llegar la etiqueta de cierre. El segundo: el programa nunca tiene delante el documento completo. Cuando se ejecuta `endElement()` para `nombre`, el parser ya ha olvidado todo lo anterior.

Con SAX nunca tienes el documento entero delante: si necesitas un dato que ya pasó, hay que volver a leerlo.

Por eso SAX conviene en dos situaciones: cuando los ficheros son muy grandes y no caben cómodamente en memoria, y cuando solo hay que extraer datos, sin modificar el documento.



Parser DOM: el árbol de nodos

DOM (Document Object Model) es el parser jerárquico de Java. En lugar de avisarte mientras lees, carga el fichero completo en memoria y lo convierte en un **árbol de nodos**: cada parte del documento XML pasa a ser un nodo del árbol. Una vez construido, puedes recorrerlo en cualquier dirección (de padres a hijos, de hijos a padres, entre hermanos) y modificarlo.

No todos los nodos son iguales. Los tipos con los que vas a trabajar son cuatro:

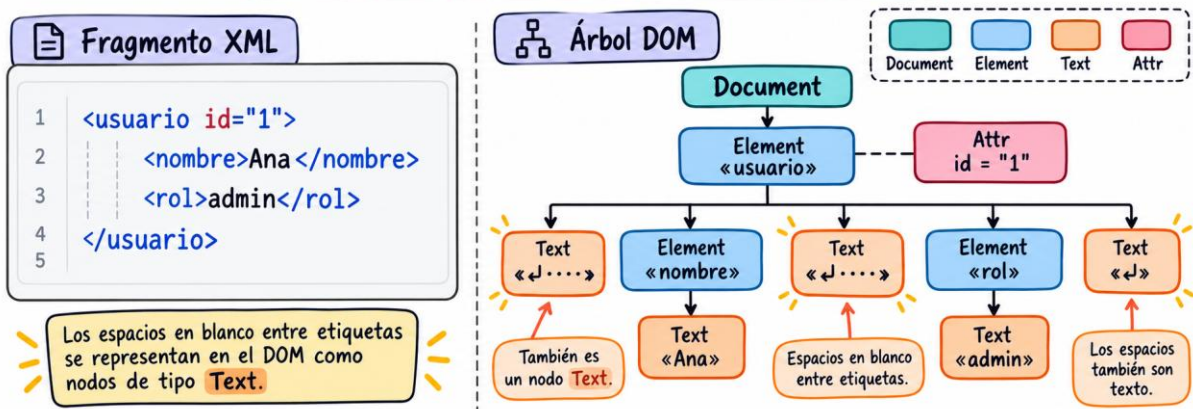
- **Document**: el documento completo. Es el punto de entrada al árbol.
- **Element**: cada etiqueta del documento, incluida la raíz. En `usuarios.xml`, `<usuarios>`, cada `<usuario>`, cada `<nombre>` y cada `<rol>` son nodos `Element`.

- **Text**: el contenido textual de una etiqueta, como Ana.
- **Attr**: cada atributo de un elemento, como `id="1"`.

Hay un detalle que conviene conocer desde el principio, porque causa el error más habitual al recorrer un DOM: **los saltos de línea y los espacios que hay entre las etiquetas también son nodos de texto**. En `usuarios.xml`, entre `<usuario id="1">` y `<nombre>` hay un salto de línea y ocho espacios, y DOM los guarda como un nodo `Text`. Si pides el primer hijo de un `<usuario>` esperando obtener `<nombre>`, lo que obtienes es ese nodo de espacios en blanco.

DOM DEL PRIMER USUARIO EN `usuarios.xml`

En DOM, los espacios entre etiquetas también son nodos.



DOM: clases y métodos

Para abrir un documento con DOM intervienen tres clases, que se usan siempre en el mismo orden:

- `DocumentBuilderFactory`: la fábrica que crea y configura el analizador.
- `DocumentBuilder`: el analizador que lee el fichero y construye el árbol.
- `Document`: el árbol resultante.

`DocumentBuilder` es una clase abstracta, así que no se puede crear con `new`: se le pide a la fábrica con `newDocumentBuilder()`. Este patrón, en el que una clase fábrica se encarga de crear los objetos, lo volverás a encontrar en SAX y en las transformaciones del apartado 5.4.

Una vez tienes el `Document`, estos son los métodos que vas a utilizar con más frecuencia:

- `getDocumentElement()`: devuelve el elemento raíz del documento.
- `getElementsByTagName(nombre)`: devuelve una lista (`NodeList`) con todos los elementos que tienen ese nombre, estén donde estén dentro del árbol.
- `getTextContent()`: devuelve el texto que contiene un elemento.
- `getAttribute(nombre)`: devuelve el valor de un atributo de un elemento.

Con todo ello, los pasos para leer un documento con DOM son siempre los mismos:

1. Crear la fábrica con `DocumentBuilderFactory.newInstance()`.
2. Pedir a la fábrica el analizador con `newDocumentBuilder()`.
3. Analizar el fichero con `parse()`, que devuelve el `Document`.
4. Normalizar el árbol con `normalize()`.

5. Recorrer los elementos que interesen, normalmente con `getElementsByTagName()`.

DOM: ejemplo de lectura

El siguiente programa lee `usuarios.xml` y muestra el identificador, el nombre y el rol de cada usuario. Para que lo encuentre, el fichero `usuarios.xml` debe estar en la carpeta raíz del proyecto de IntelliJ, que es la carpeta desde la que se ejecuta el programa.

```
import java.io.File;
import java.io.IOException;
import javax.xml.parsers.DocumentBuilder;
import javax.xml.parsers.DocumentBuilderFactory;
import javax.xml.parsers.ParserConfigurationException;
import org.w3c.dom.Document;
import org.w3c.dom.Element;
import org.w3c.dom.NodeList;
import org.xml.sax.SAXException;

public class LectorDOM {
    public static void main(String[] args) {
        try {
            // Pasos 1 y 2: fábrica y analizador
            DocumentBuilderFactory fabrica = DocumentBuilderFactory.newInstance();
            DocumentBuilder analizador = fabrica.newDocumentBuilder();

            // Pasos 3 y 4: análisis del fichero y normalización del árbol
            Document doc = analizador.parse(new File("usuarios.xml"));
            doc.getDocumentElement().normalize();
            System.out.println("Raíz: " + doc.getDocumentElement().getNodeName());

            // Paso 5: recorrido de los elementos <usuario>
            NodeList usuarios = doc.getElementsByTagName("usuario");
            for (int i = 0; i < usuarios.getLength(); i++) {
                Element u = (Element) usuarios.item(i);
                String id = u.getAttribute("id");
                String nombre = u.getElementsByTagName("nombre")
                    .item(0).getTextContent();
                String rol = u.getElementsByTagName("rol")
                    .item(0).getTextContent();
                System.out.println(id + ": " + nombre + " (" + rol + ")");
            }
        } catch (ParserConfigurationException | SAXException | IOException e) {
            System.err.println("Error al leer el XML: " + e.getMessage());
        }
    }
}
```

Y esta es la salida:

```
1  Raíz: usuarios
2  1: Ana (admin)
3  2: Íñigo (editor)
```

Hay dos cosas que conviene controlar en este código:

- **Las excepciones.** `newDocumentBuilder()` puede lanzar `ParserConfigurationException`, y `parse()` puede lanzar `IOException` (si el fichero no existe o no se puede leer) y `SAXException` (si el documento no está bien formado). Las tres se capturan en un único bloque `catch`, con una sintaxis que estudiarás en el apartado 6.3.
- **La normalización.** `normalize()` une los nodos de texto que hayan quedado partidos en varios trozos y evita sorpresas al recorrer el árbol.

Observa también cómo se evita el problema de los espacios en blanco: en lugar de navegar por los hijos con `getFirstChild()`, el programa busca directamente los elementos por su nombre con `getElementsByTagName()`, que solo devuelve nodos `Element`.

Recorre siempre con `getElementsByTagName()` antes que con `getFirstChild()`: te saltas los nodos de espacios en blanco.

Para practicar lo visto en este apartado, realiza la Actividad 6 (RA1_A6), «Lectura de XML con DOM», disponible en el Aula Virtual.

5.3 Vinculación o binding (JAXB)

Con DOM y SAX tu programa trabaja con etiquetas, nodos y eventos. Funciona, pero obliga a escribir mucho código para algo tan habitual como convertir un `<usuario>` en un objeto `Usuario`. El **binding** evita ese trabajo: enlaza el documento XML con clases de Java para que puedas trabajar directamente con objetos.

Binding frente a parser

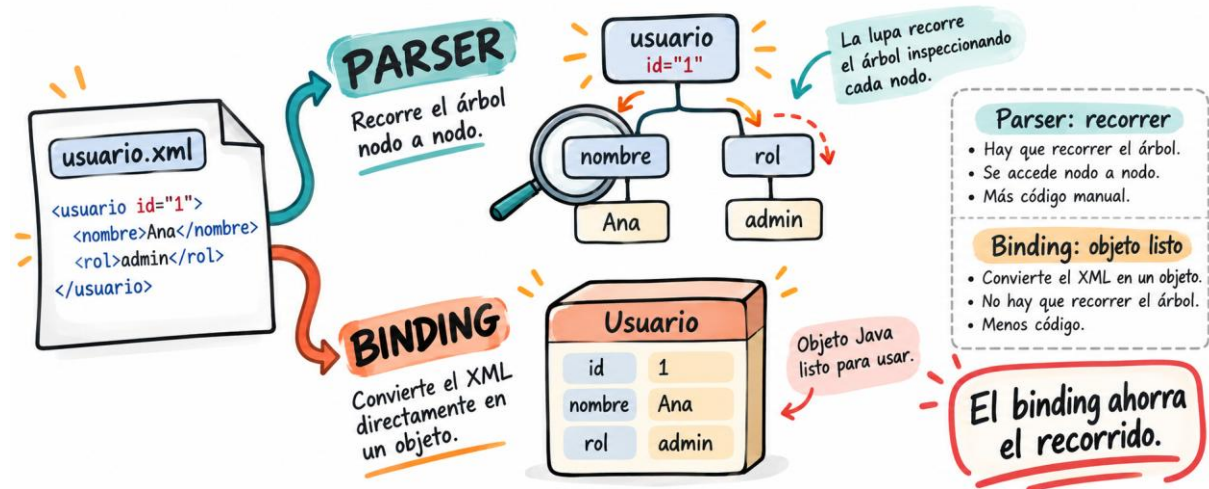
La diferencia entre las dos formas de trabajar se entiende mejor comparándolas punto por punto:

	Parser	Binding
Propósito	Leer y analizar un documento estructurado y crear una representación interna.	Relacionar los datos con objetos del lenguaje para manipularlos directamente como clases.
Resultado	Un árbol de nodos (DOM) o una serie de eventos (SAX) que representa el documento.	Objetos cuyos campos contienen los valores del documento.
Nivel de abstracción	Bajo: tienes que recorrer y manipular el documento de forma explícita.	Alto: trabajas con objetos y el XML desaparece de tu código.
Ejemplos	DOM, SAX y StAX para XML; parsers JSON para JSON.	JAXB para XML; Jackson o Gson para JSON.

Con un parser recorres el documento. Con binding trabajas con objetos y el XML desaparece de tu código.

¿Cuándo compensa cada uno? El binding es la opción natural cuando el XML refleja objetos que ya existen en tu aplicación, como usuarios, productos o pedidos. Un parser sigue siendo

mejor cuando el documento no encaja con ninguna clase o cuando solo necesitas extraer una parte pequeña de él.



JAXB en el proyecto: dependencias y anotaciones

En Java, el binding se realiza con **JAXB** (*Java Architecture for XML Binding*), una API estándar que permite convertir documentos XML en objetos Java y objetos Java en documentos XML. Durante años formó parte del JDK, pero se eliminó en Java 11, así que hoy hay que añadirla al proyecto como dependencia externa. Se necesitan dos piezas: la **API** (Jakarta XML Binding), que contiene las clases y anotaciones que usa tu código, y la **implementación** (JAXB Runtime), que es la que hace el trabajo al ejecutar.

En un proyecto Maven, basta con añadir estas dos dependencias al fichero `pom.xml`, dentro de la etiqueta `<dependencies>` (si no existe, créala dentro de `<project>`):

```

1 <dependencies>
2   <dependency>
3     <groupId>jakarta.xml.bind</groupId>
4     <artifactId>jakarta.xml.bind-api</artifactId>
5     <version>4.0.5</version>
6     <scope>compile</scope>
7   </dependency>
8   <dependency>
9     <groupId>org.glassfish.jaxb</groupId>
10    <artifactId>jaxb-runtime</artifactId>
11    <version>4.0.9</version>
12    <scope>compile</scope>
13  </dependency>
14 </dependencies>
```

Después de guardar el `pom.xml`, pulsa el botón de recarga de Maven que aparece en IntelliJ (o usa la ventana Maven y el botón *Reload All Maven Projects*) para que descargue las librerías. Hasta que no lo hagas, las anotaciones de JAXB aparecerán en rojo en tu código.

Con las dependencias en su sitio, el siguiente paso es preparar la clase. JAXB necesita saber qué parte del XML corresponde a cada campo, y eso se le indica con **anotaciones**, que son las marcas que empiezan por `@` y se colocan delante de la clase o de sus campos. Esta es la clase `Usuario`, preparada para el documento de usuarios del apartado 5.1:

```
import jakarta.xml.bind.annotation.XmlAccessType;
import jakarta.xml.bind.annotation.XmlAccessorType;
import jakarta.xml.bind.annotation.XmlAttribute;
import jakarta.xml.bind.annotation.XmlElement;
import jakarta.xml.bind.annotation.XmlRootElement;

@XmlRootElement(name = "usuario")
@XmlAccessorType(XmlAccessType.FIELD)
public class Usuario {

    @XmlAttribute
    private int id;

    @XmlElement
    private String nombre;

    @XmlElement
    private String rol;

    public Usuario() {} // constructor vacío: obligatorio para JAXB

    public Usuario(int id, String nombre, String rol) {
        this.id = id;
        this.nombre = nombre;
        this.rol = rol;
    }

    public int getId() { return id; }
    public String getNombre() { return nombre; }
    public String getRol() { return rol; }

    @Override
    public String toString() {
        return id + ": " + nombre + " (" + rol + ")";
    }
}
```

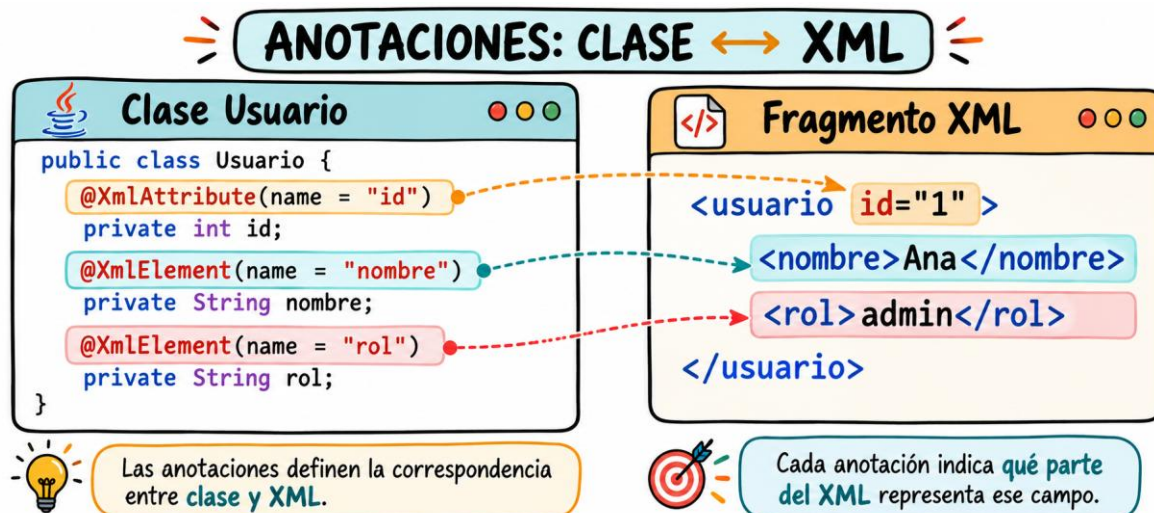
Cada anotación tiene una función concreta:

Anotación	Dónde va	Qué indica
@XmlRootElement	Delante de la clase	Que la clase puede ser el elemento raíz de un documento. Con name se indica el nombre de la etiqueta.
@XmlAccessorType	Delante de la clase	Que JAXB lea y escriba directamente los campos (FIELD), aunque sean privados y no tengan setters.
@XmlAttribute	Delante de un campo	Que el campo se guarda como atributo de la etiqueta: <usuario id="1">.
@XmlElement	Delante de un campo	Que el campo se guarda como elemento hijo: <nombre>Ana</nombre>.

Por defecto, el nombre de cada atributo y de cada elemento es el nombre del campo. Si en el XML fuera distinto, se indicaría con `name`, por ejemplo `@XmlElement(name = "nombreCompleto")`.

Queda un último requisito, que no depende de ninguna anotación: la clase debe tener un **constructor vacío**. Cuando JAXB convierte un XML en objeto, primero crea un objeto vacío y después rellena sus campos, y para crearlo necesita ese constructor. Si la clase solo tiene constructores con parámetros, el programa compila sin problemas, pero falla al ejecutarse.

Sin constructor vacío, JAXB no puede reconstruir el objeto y falla en tiempo de ejecución, no al compilar.



5.3.1 Marshalling: del objeto Java al XML

El **marshalling** (también llamado serialización) es el proceso de convertir un objeto Java en un documento XML. Se realiza en tres pasos:

- 1. Crear un contexto JAXB.** La clase `JAXBContext` representa el entorno de binding para las clases que vas a procesar. Se crea con `JAXBContext.newInstance()`, indicando la clase: `JAXBContext.newInstance(Usuario.class)`.
- 2. Crear un marshaller.** Se le pide al contexto con `createMarshaller()`. Es el objeto que gestiona la conversión de objeto a XML.
- 3. Serializar el objeto.** El método `marshal()` recibe el objeto y el destino, que puede ser un fichero (`File`), la consola (`System.out`) o cualquier flujo de salida (`OutputStream`).

El siguiente programa crea un usuario y lo guarda en `usuario.xml`. Para comprobar el resultado sin abrir el fichero, lo escribe también en la consola:

```
import jakarta.xml.bind.JAXBContext;
import jakarta.xml.bind.JAXBException;
import jakarta.xml.bind.Marshaller;
import java.io.File;

public class EscribirUsuario {
    public static void main(String[] args) {
        Usuario usuario = new Usuario(1, "Ana", "admin");
        try {
```

```
// 1. Contexto JAXB para la clase Usuario
JAXBContext contexto = JAXBContext.newInstance(Usuario.class);
// 2. Marshaller a partir del contexto
Marshaller marshaller = contexto.createMarshaller();
// 3. Salida con saltos de línea y sangrado
marshaller.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true);
// 4. Serialización: a un fichero y, para comprobarlo, a la consola
marshaller.marshal(usuario, new File("usuario.xml"));
marshaller.marshal(usuario, System.out);
} catch (JAXBException e) {
    System.err.println("Error al generar el XML: " + e.getMessage());
}
}
```

Esta es la salida por consola, que coincide con el contenido de `usuario.xml`:

```
1 <?xml version="1.0" encoding="UTF-8" standalone="yes"?>
2 <usuario id="1">
3     <nombre>Ana</nombre>
4     <rol>admin</rol>
5 </usuario>
```

Hay tres cosas que conviene controlar:

- **JAXBException.** Todo el proceso puede lanzarla, y es una excepción comprobada: el compilador te obliga a capturarla o a declararla (lo verás en el apartado 6).
- **JAXB_FORMATTED_OUTPUT.** Esta propiedad solo afecta a la legibilidad, no al contenido. Sin ella, el XML sale en una sola línea: es válido, pero muy difícil de leer y de corregir.
- **La declaración.** JAXB añade `standalone="yes"` a la primera línea, que indica que el documento no depende de ningún fichero externo. No tienes que hacer nada con ella.

Para practicar el marshalling, realiza la Actividad 7 (RA1_A7) del Aula Virtual.



5.3.2 Unmarshalling: del XML al objeto Java

El **unmarshalling** (o deserialización) es el proceso inverso: leer un documento XML y convertirlo en un objeto Java con el que tu programa puede trabajar de forma nativa. Los pasos son equivalentes a los del marshalling:

1. **Crear un contexto JAXB**, exactamente igual que antes.
2. **Crear un unmarshaller**. Se le pide al contexto con `createUnmarshaller()`.
3. **Deserializar el fichero**. El método `unmarshal()` lee el XML y devuelve el objeto. Como lo devuelve con el tipo genérico `Object`, hay que convertirlo al tipo esperado con un `cast`: `(Usuario)`.

El siguiente programa lee el `usuario.xml` que generó el ejemplo anterior:

```
import jakarta.xml.bind.JAXBContext;
import jakarta.xml.bind.JAXBException;
import jakarta.xml.bind.Unmarshaller;
import java.io.File;

public class LeerUsuario {
    public static void main(String[] args) {
        try {
            // 1. Contexto JAXB para la clase Usuario
            JAXBContext contexto = JAXBContext.newInstance(Usuario.class);
            // 2. Unmarshaller a partir del contexto
            Unmarshaller unmarshaller = contexto.createUnmarshaller();
            // 3. Deserialización: unmarshal devuelve Object, hay que convertirlo
            Usuario usuario = (Usuario) unmarshaller.unmarshal(
                new File("usuario.xml"));
            System.out.println(usuario.getNombre());
            System.out.println(usuario);
        } catch (JAXBException e) {
            System.err.println("Error al leer el XML: " + e.getMessage());
        }
    }
}
```

Salida:

```
1 Ana
2 1: Ana (admin)
```

Si el unmarshalling no da el resultado esperado, casi siempre se debe a uno de estos dos errores:

- **Los campos llegan a null.** El XML no coincide con las anotaciones. Si en el fichero pone `<Nombre>` y el campo se llama `nombre`, JAXB no los relaciona, porque XML distingue mayúsculas de minúsculas, y el campo se queda vacío sin avisar.
- **Salta una `UnmarshallerException`.** La etiqueta raíz del documento no coincide con la indicada en `@XmlRootElement`. Por ejemplo, intentar leer `usuarios.xml` (raíz `<usuarios>`) con el contexto de la clase `Usuario` (raíz `<usuario>`).

Un campo a null no siempre es un fallo del fichero: revisa primero que el nombre de la etiqueta y el de la anotación sean idénticos, mayúsculas incluidas.

Precisamente ese segundo error plantea una pregunta: ¿cómo se lee el documento `usuarios.xml` completo, que contiene varios usuarios? La solución es crear una clase que represente la raíz `<usuarios>` y que guarde una lista de objetos `Usuario`. A este tipo de clase se la suele llamar **clase envoltorio**:

```
import jakarta.xml.bind.annotation.XmlAccessType;
import jakarta.xml.bind.annotation.XmlAccessorType;
import jakarta.xml.bind.annotation.XmlElement;
import jakarta.xml.bind.annotation.XmlRootElement;
import java.util.ArrayList;
import java.util.List;

@XmlRootElement(name = "usuarios")
@XmlAccessorType(XmlAccessType.FIELD)
public class Usuarios {

    // cada elemento de la lista se escribe como un <usuario>
    @XmlElement(name = "usuario")
    private List<Usuario> lista = new ArrayList<>();

    public Usuarios() {}

    public List<Usuario> getLista() { return lista; }
}
```

La anotación `@XmlElement(name = "usuario")` sobre la lista indica que cada objeto de la lista corresponde a un elemento `<usuario>`. Con esta clase, leer el documento completo solo requiere cambiar la clase del contexto:

```
JAXBContext contexto = JAXBContext.newInstance(Usuarios.class);
Unmarshaller unmarshaller = contexto.createUnmarshaller();
Usuarios usuarios = (Usuarios) unmarshaller.unmarshal(new File("usuarios.xml"));
for (Usuario u : usuarios.getLista()) {
    System.out.println(u);
}
```

La salida muestra los dos usuarios del fichero, «1: Ana (admin)» y «2: Íñigo (editor)». El mismo esquema sirve para el marshalling: si rellenas un objeto `Usuarios` con varios usuarios y lo serializas, obtienes un documento como `usuarios.xml`.

Para practicar el unmarshalling, realiza la Actividad 8 (RA1_A8) del Aula Virtual.

5.4 Conversión de ficheros XML a otros formatos

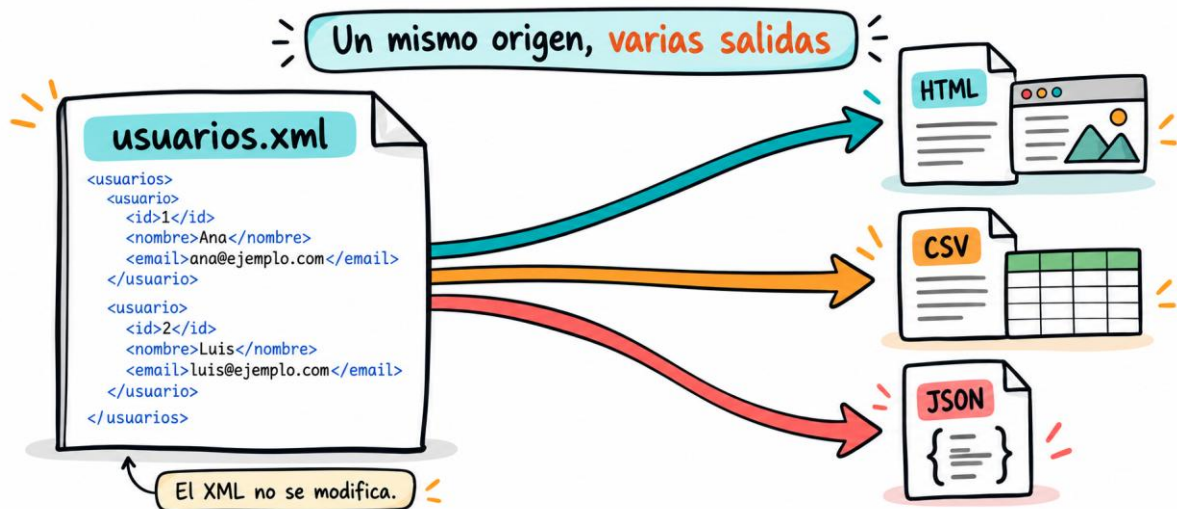
XML es muy bueno para guardar e intercambiar datos, pero no para todo. Una persona no quiere leer etiquetas: quiere ver una tabla en el navegador o abrir los datos en una hoja de cálculo. Por eso es habitual convertir un documento XML a otros formatos según el uso que se le vaya a dar:

- **HTML**, para mostrar los datos en un navegador.
- **CSV**, para analizarlos en una hoja de cálculo como Excel o LibreOffice Calc.
- **JSON**, para enviarlos a una aplicación web.

Para hacer la conversión hay dos caminos:

- **Con una plantilla XSLT.** Describes cómo debe ser el resultado y un motor de transformación hace el trabajo. No hay que programar la conversión, solo escribir la plantilla. Es el camino que vas a estudiar en este apartado.
- **Con código Java y una librería.** Lees el XML (con DOM o con JAXB) y generas tú el nuevo formato. Da más control, pero supone más trabajo. Es el camino habitual para JSON, con librerías como Jackson o Gson.

Los mismos datos, varias presentaciones. El XML original no se toca: se genera un fichero nuevo.



XSL y XSLT: la plantilla de transformación

Para describir cómo se transforma un documento XML se utilizan tres tecnologías relacionadas entre sí:

- **XSL (eXtensible Stylesheet Language):** es la familia de lenguajes para transformar y presentar documentos XML.
- **XSLT (XSL Transformations):** es la parte de XSL que define cómo se genera el documento de salida. Una plantilla XSLT es la «receta» de la transformación.
- **XPath:** es la forma de seleccionar los nodos del XML que se quieren usar. Por ejemplo, `usuarios/usuario` selecciona todos los usuarios, `nombre` selecciona el elemento nombre y `@id` selecciona el atributo id.

Esta plantilla, `usuarios.xsl`, convierte `usuarios.xml` en una página web con la lista de usuarios. Es la plantilla de las transparencias, completada con la estructura de una página HTML para que el navegador la muestre correctamente:

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <xsl:stylesheet version="1.0"
3   xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
4   <xsl:output method="html" encoding="UTF-8"/>
5   <xsl:template match="/">
6     <html>
7       <head>
8         <title>Usuarios</title>
9       </head>
10      <body>
11        <h1>Usuarios</h1>
12        <ul>
13          <xsl:for-each select="usuarios/usuario">
```

```

14         <li>
15             <xsl:value-of select="nombre"/>
16             <xsl:text> (</xsl:text>
17             <xsl:value-of select="rol"/>
18             <xsl:text>)</xsl:text>
19         </li>
20     </xsl:for-each>
21 </ul>
22 </body>
23 </html>
24 </xsl:template>
25 </xsl:stylesheet>

```

Léela por partes:

- `<xsl:stylesheet>` es la raíz de la plantilla. El atributo `xmlns:xsl` declara que las etiquetas que empiezan por `xsl:` son instrucciones de transformación; el resto (`<html>`, `<ul>`, `<li>`...) se copia tal cual en el resultado.
- `<xsl:output method="html">` indica el tipo de documento que se va a generar.
- `<xsl:template match="/">` define qué hacer al procesar el documento desde su raíz. Todo lo que hay dentro es lo que se escribe en la salida.
- `<xsl:for-each select="usuarios/usuario">` repite su contenido una vez por cada usuario.
- `<xsl:value-of select="nombre"/>` escribe el texto del elemento `nombre` del usuario que se está procesando.
- `<xsl:text>` escribe un texto fijo, como los paréntesis que rodean al rol.

El resultado, abierto en el navegador, es una página con el título «Usuarios» y una lista con «Ana (admin)» e «Íñigo (editor)».

La plantilla es a su vez un documento XML: si no está bien formada, la transformación falla antes de empezar.

Cambiando la plantilla cambia el formato de salida. Esta otra, `usuarios_csv.xsl`, genera un fichero CSV con los campos separados por punto y coma. La diferencia principal es `method="text"`, que indica que la salida es texto plano. Los fragmentos de texto fijo, como el punto y coma, se escriben con `<xsl:text>`, y `
` representa un salto de línea:

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <xsl:stylesheet version="1.0"
3     xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
4     <xsl:output method="text" encoding="UTF-8"/>
5     <xsl:template match="/">
6         <xsl:text>id;nombre;rol&#10;</xsl:text>
7         <xsl:for-each select="usuarios/usuario">
8             <xsl:value-of select="@id"/>
9             <xsl:text>;</xsl:text>
10            <xsl:value-of select="nombre"/>
11            <xsl:text>;</xsl:text>
12            <xsl:value-of select="rol"/>
13            <xsl:text>&#10;</xsl:text>
14        </xsl:for-each>
15    </xsl:template>
16 </xsl:stylesheet>

```

Resultado de aplicarla sobre `usuarios.xml`:

```

1 id;nombre;rol
2 1;Ana;admin
3 2;Íñigo;editor

```



La transformación en Java

Java incluye en el JDK todo lo necesario para aplicar plantillas XSLT, dentro del paquete `javax.xml.transform`. No hace falta añadir ninguna dependencia al proyecto. Intervienen estas clases:

Clase	Función
TransformerFactory	Crea el transformador a partir de la plantilla.
Transformer	Aplica la plantilla sobre un documento. Permite configurar propiedades de salida, como el sangrado.
StreamSource	Representa la entrada: el fichero XML o el fichero de la plantilla.
StreamResult	Representa la salida: el fichero que se va a generar.

Los pasos para hacer una transformación son tres:

- 1. Cargar la plantilla.** Con `TransformerFactory` se crea un `Transformer` a partir del fichero XSLT.
- 2. Configurar el transformador,** si hace falta ajustar alguna propiedad de la salida.
- 3. Aplicar la transformación** con el método `transform()`, que recibe el XML de entrada y el destino de la salida.

El siguiente programa aplica la plantilla `usuarios.xsl` sobre `usuarios.xml` y genera `usuarios.html`. Los tres ficheros deben estar en la carpeta raíz del proyecto:

```
import java.io.File;
import javax.xml.transform.Transformer;
import javax.xml.transform.TransformerException;
import javax.xml.transform.TransformerFactory;
import javax.xml.transform.stream.StreamResult;
import javax.xml.transform.stream.StreamSource;

public class XmlAHtml {
```

```
public static void main(String[] args) {
    try {
        // 1. Cargar la plantilla XSLT
        TransformerFactory fabrica = TransformerFactory.newInstance();
        Transformer t = fabrica.newTransformer(
            new StreamSource(new File("usuarios.xml")));

        // 2 y 3. Aplicar la plantilla: XML de entrada y fichero de salida
        t.transform(new StreamSource(new File("usuarios.xml")),
            new StreamResult(new File("usuarios.html")));
        System.out.println("Fichero usuarios.html generado.");
    } catch (TransformerException e) {
        System.err.println("Error en la transformación: " + e.getMessage());
    }
}
}
```

Para generar el CSV basta con cambiar los nombres de los ficheros: `usuarios_csv.xml` como plantilla y `usuarios.csv` como salida. El código Java es el mismo; lo único que cambia es la plantilla. Si la plantilla tiene algún error, `newTransformer()` lanza una `TransformerConfigurationException`, que es un tipo de `TransformerException` y, por tanto, también la captura el bloque `catch`.

La clase `Transformer` tiene además otro uso muy práctico. Si se crea **sin plantilla**, con `newTransformer()` sin parámetros, no transforma nada: copia el documento tal cual. Es la forma estándar de guardar en un fichero un árbol DOM que has modificado en memoria. Para ello se usa `DOMSource` como entrada, en lugar de `StreamSource`. El siguiente programa lee `usuarios.xml` con DOM, añade un tercer usuario y guarda el resultado en un fichero nuevo:

```
import org.w3c.dom.Element;

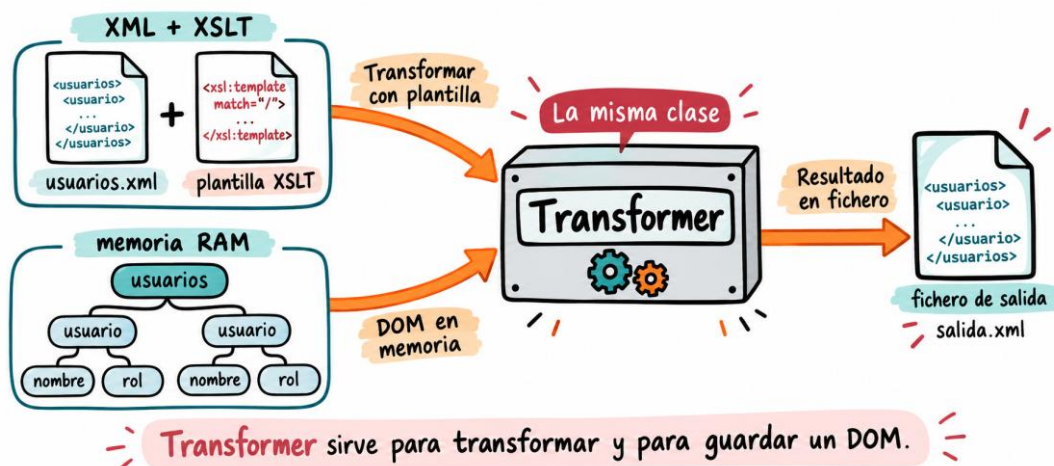
public class AnadirUsuario {
    // throws Exception para simplificar; la gestión correcta, en el apartado 6
    public static void main(String[] args) throws Exception {
        Document doc = DocumentBuilderFactory.newInstance()
            .newDocumentBuilder().parse(new File("usuarios.xml"));

        // Se crea el nuevo <usuario> con sus dos hijos y se añade a la raíz
        Element nuevo = doc.createElement("usuario");
        nuevo.setAttribute("id", "3");
        Element nombre = doc.createElement("nombre");
        nombre.setTextContent("Lucia");
        Element rol = doc.createElement("rol");
        rol.setTextContent("lector");
        nuevo.appendChild(nombre);
        nuevo.appendChild(rol);
        doc.getDocumentElement().appendChild(nuevo);

        // Transformer sin plantilla: copia el árbol tal cual a un fichero
        Transformer t = TransformerFactory.newInstance().newTransformer();
        t.transform(new DOMSource(doc),
            new StreamResult(new File("usuarios_nuevo.xml")));
    }
}
```

El programa guarda el resultado en `usuarios_nuevo.xml` para no sobrescribir el original mientras haces pruebas. Si abres ese fichero verás el tercer usuario al final, dentro de `<usuarios>`, escrito en una sola línea: como el nuevo elemento se ha creado en memoria, no tiene saltos de línea ni espacios entre sus etiquetas. Por el mismo motivo, la declaración y la etiqueta `<usuarios>` aparecen juntas en la primera línea. Sigue siendo un documento bien formado y perfectamente legible por cualquier programa.

Sin plantilla, Transformer copia el documento tal cual: es la forma estándar de volcar a fichero un DOM que has modificado.



6 Manejo de excepciones

A lo largo de esta unidad has visto que casi todas las operaciones con ficheros pueden fallar: el fichero no existe, no tienes permiso para leerlo, el XML está mal formado... Java no deja que ignores esos fallos. En este apartado vas a aprender qué son las excepciones, cómo capturarlas, cómo tratar de forma distinta cada tipo de error y cómo trasladar la responsabilidad de gestionarlas a quien llama a un método.

6.1 Introducción a las excepciones en Java

¿Qué son las excepciones?

Una **excepción** es un evento que ocurre durante la ejecución de un programa y que interrumpe su flujo normal. Surge cuando el programa se encuentra con una situación inesperada que no puede resolver por sí mismo: dividir un número entre cero, acceder a una posición que no existe en un array, abrir un fichero que no está en el disco o utilizar un objeto que vale `null`.

Observa qué ocurre cuando un programa se encuentra con una de estas situaciones y nadie la gestiona:

```
public class EjemploDivision {
    public static void main(String[] args) {
        int resultado = 10 / 0;
        System.out.println("Resultado: " + resultado);
        System.out.println("Fin del programa.");
    }
}
```

Al ejecutarlo, el programa se detiene en la línea 3 y no llega a mostrar ni el resultado ni el mensaje final. En su lugar, la consola muestra esto:

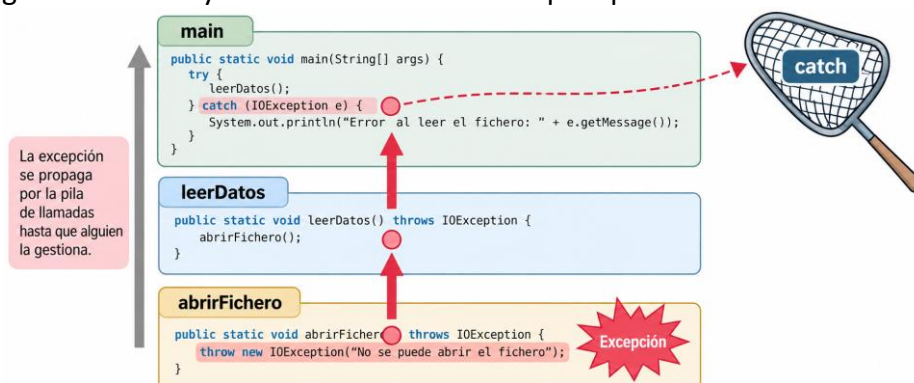
```
1 Exception in thread "main" java.lang.ArithmeticException: / by zero
2 at EjemploDivision.main(EjemploDivision.java:3)
```

Este mensaje se llama **traza de la pila** (*stack trace*) y es muy útil para depurar: indica el tipo de excepción (`ArithmeticException`), una descripción del problema (`/ by zero`) y el lugar exacto donde se produjo (línea 3 del fichero `EjemploDivision.java`).

¿Cómo gestiona Java las excepciones?

Java proporciona un mecanismo estructurado para manejar estas situaciones, basado en la clase `Exception` y su jerarquía de subclases. El mecanismo funciona en dos tiempos:

- **Creación del objeto de excepción.** Cuando ocurre un error durante la ejecución, Java crea un objeto que contiene los detalles del problema: de qué tipo es, qué mensaje lo describe y en qué punto del programa se produjo. Las excepciones, por tanto, son objetos.
- **Propagación.** Ese objeto se «lanza» hacia atrás por la pila de llamadas: primero al método donde ocurrió el error, luego al método que lo llamó, y así sucesivamente, hasta que algún método lo gestione. Si llega hasta `main` y nadie lo gestiona, el programa termina y se muestra la traza de la pila que acabas de ver.



Importancia de la gestión de excepciones

Gestionar las excepciones no es un trámite para que el programa compile, sino una parte esencial de cualquier aplicación profesional. Aporta tres ventajas:

- **Previene errores críticos.** Evita que un fallo puntual detenga toda la aplicación. Si un fichero de configuración no existe, el programa puede avisar y usar valores por defecto en lugar de terminar.
- **Mejora la robustez.** Los fallos inesperados se manejan de forma controlada, lo que aumenta la fiabilidad del programa.
- **Facilita la depuración.** Capturar una excepción permite mostrar mensajes claros, que ayudan a localizar y resolver el problema.

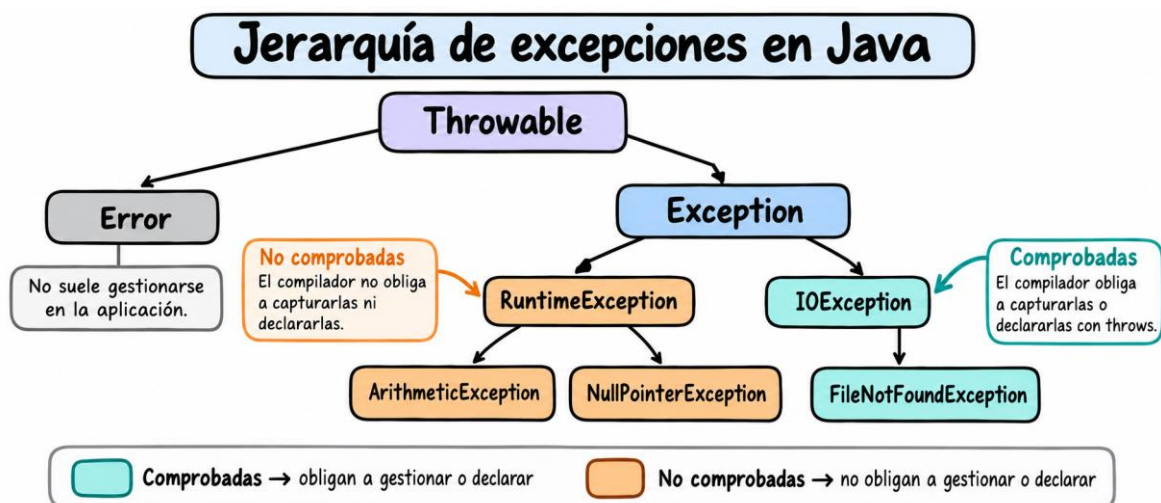
Excepciones comprobadas y no comprobadas

No todas las excepciones reciben el mismo trato por parte del compilador. Java las divide en dos grandes grupos:

- **Excepciones comprobadas** (*checked*). El compilador te obliga a gestionarlas, ya sea capturándolas con `try-catch` o declarándolas con `throws`. Si no lo haces, el programa no compila. Son las que representan fallos externos al programa, que pueden ocurrir aunque el código sea correcto: `IOException` en las operaciones con ficheros o `SQLException` en el acceso a bases de datos. Por eso su gestión es esencial en este módulo.
- **Excepciones no comprobadas** (*unchecked*). Heredan de la clase `RuntimeException` y el compilador no obliga a gestionarlas. Suelen deberse a errores de programación, como `NullPointerException` o `ArrayIndexOutOfBoundsException`, que se deberían evitar corrigiendo el código. Gestionarlas no es obligatorio, pero puede mejorar la robustez del programa.

Seguramente ya te has encontrado con las comprobadas sin saberlo. Cuando en las actividades de esta unidad escribiste `new FileReader("fichero.txt")`, IntelliJ lo subrayó en rojo con el mensaje *Unhandled exception: java.io.FileNotFoundException*. Es el compilador exigiéndote que decidas qué hacer si el fichero no existe.

	Comprobadas	No comprobadas
¿Obliga el compilador?	Sí: try-catch o throws	No
Heredan de	Exception (pero no de RuntimeException)	RuntimeException
Causa habitual	Fallos externos: disco, red, base de datos	Errores de programación
Ejemplos	IOException, FileNotFoundException, SQLException, JAXBException	ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException



6.2 Captura y gestión de excepciones

Cuando una excepción no se gestiona, el programa se detiene bruscamente y muestra un mensaje de error, como has visto en el ejemplo de la división. Para evitarlo, Java permite **capturar** las excepciones con bloques `try-catch` y controlar qué hace el programa cuando ocurre el error, de forma que pueda reaccionar de manera apropiada y continuar su ejecución.

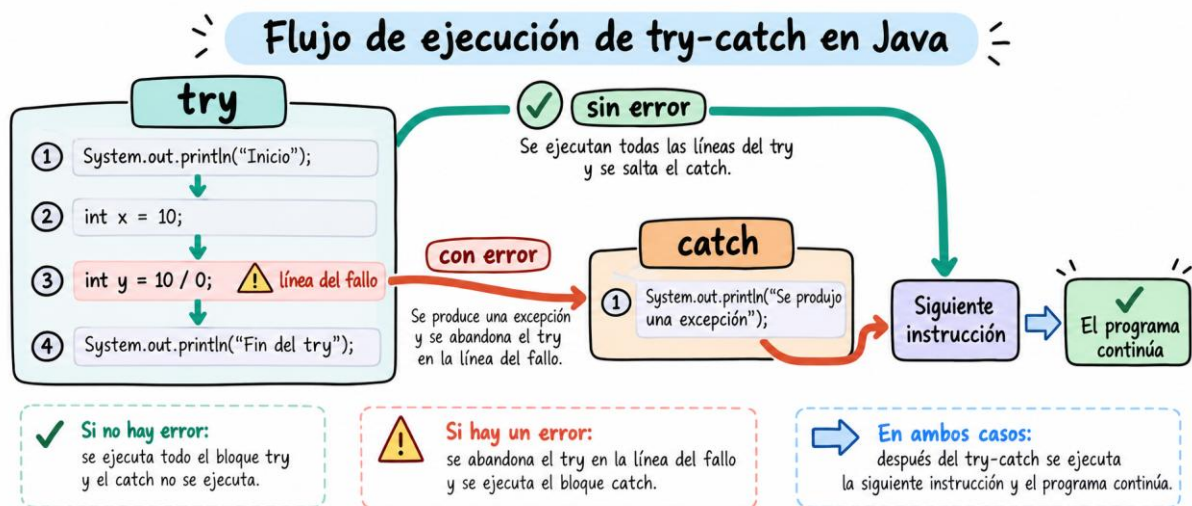
Estructura del bloque `try-catch`

```
try {
    // Código que puede lanzar una excepción
} catch (TipoDeExcepcion e) {
    // Código para gestionar la excepción
}
```

La estructura funciona así:

- Bloque `try`.** Contiene el código que puede generar una excepción.
- Excepción lanzada.** Si durante la ejecución del bloque `try` se produce una excepción, el resto de instrucciones del `try` no se ejecutan: el flujo salta directamente al bloque `catch`.
- Bloque `catch`.** Captura y gestiona la excepción del tipo indicado entre paréntesis. La variable `e` es el objeto de la excepción, con toda la información sobre el error.

Si en el bloque `try` no se produce ninguna excepción, el bloque `catch` se salta por completo. En los dos casos, el programa continúa con la instrucción que hay después del `catch`.



Ejemplo de captura de excepciones

El siguiente ejemplo captura la `ArithmeticException` que se produce al dividir entre cero. A diferencia del ejemplo anterior, el programa no se detiene: muestra un mensaje y continúa con su ejecución normal.

```
public class EjemploTryCatch {
    public static void main(String[] args) {
        try {
            int resultado = 10 / 0; // Esto provocará una ArithmeticException
        } catch (ArithmeticException e) {
            System.out.println("Error: División por cero.");
        }
    }
}
```

```

    System.out.println("El resultado es: " + resultado);
} catch (ArithmeticException e) {
    System.err.println("Error: No se puede dividir por cero.");
}
System.out.println("Fin del programa.");
}
}

```

Salida del programa:

```

1 Error: No se puede dividir por cero.
2 Fin del programa.

```

Fíjate en que la línea que muestra el resultado no llega a ejecutarse: la excepción se produce justo antes y el flujo salta al `catch`. En cambio, «Fin del programa» sí se muestra, porque está después del bloque.

Recomendación: utiliza `System.err` en lugar de `System.out` para mostrar los mensajes de error. `System.err` es la salida destinada específicamente a los errores, y en la consola de IntelliJ se muestra en rojo, lo que ayuda a distinguirlos.

Ejemplo con ficheros

En esta unidad, las excepciones que vas a capturar con más frecuencia son las de entrada y salida. El siguiente programa intenta leer la primera línea de un fichero que no existe:

```

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class LeerFichero {
    public static void main(String[] args) {
        try {
            BufferedReader br = new BufferedReader(new FileReader("no_existe.txt"));
            System.out.println(br.readLine());
            br.close();
        } catch (IOException e) {
            System.err.println("Error: " + e.getMessage());
        }
        System.out.println("Fin del programa.");
    }
}

```

Salida del programa en Windows:

```

1 Error: no_existe.txt (El sistema no puede encontrar el archivo especificado)
2 Fin del programa.

```

El método `getMessage()` del objeto de la excepción devuelve la descripción del error, que en este caso incluye el nombre del fichero y el motivo del fallo. Es mucho más útil que un mensaje genérico, porque dice exactamente qué ha pasado. Si durante el desarrollo necesitas más detalle, `e.printStackTrace()` muestra la traza completa de la pila.

Este ejemplo tiene todavía un problema: si `readLine()` fallara, la instrucción `br.close()` no llegaría a ejecutarse y el fichero quedaría abierto. Lo resolverás en el apartado 6.4 con `try-with-resources`.

6.3 Gestión diferenciada de distintos tipos de excepciones

Un mismo bloque de código puede generar excepciones de tipos distintos, y no siempre conviene reaccionar igual ante todas. Que un fichero no exista no es lo mismo que un error del disco a mitad de la lectura: en el primer caso puedes pedir otro nombre de fichero; en el segundo, quizá solo quepa avisar y terminar.

Múltiples bloques catch

Java permite escribir varios bloques `catch` detrás de un mismo `try`. Cada bloque se especializa en capturar un tipo concreto de excepción, y cuando se lanza una excepción, solo la gestiona el bloque correspondiente a su tipo. Así se consigue un control mucho más preciso sobre el flujo del programa.

En el siguiente ejemplo se gestionan dos tipos de excepciones: una `ArrayIndexOutOfBoundsException` (acceso fuera de los límites de un array) y una `ArithmeticException` (división entre cero).

```
public class EjemploMultiplesCatch {
    public static void main(String[] args) {
        try {
            int[] numeros = {1, 2, 3};
            System.out.println(numeros[3]); // ArrayIndexOutOfBoundsException
            int resultado = 10 / 0; // ArithmeticException
        } catch (ArrayIndexOutOfBoundsException e) {
            System.err.println("Error: Acceso fuera de los límites del array.");
        } catch (ArithmeticException e) {
            System.err.println("Error: No se puede dividir por cero.");
        }
        System.out.println("Fin del programa.");
    }
}
```

Si el error se produce en la primera instrucción que puede fallar (el acceso a `numeros[3]`, que no existe), lo captura el primer bloque `catch`. Como el flujo salta al `catch` en cuanto se produce la primera excepción, la división entre cero ya no llega a ejecutarse. Si el array tuviera cuatro elementos, el acceso funcionaría, el error se produciría en la división y lo capturaría el segundo bloque. En cada ejecución, como mucho, se ejecuta un único bloque `catch`.

Aplicado a ficheros, la gestión diferenciada permite distinguir si el fichero no existe o si ha fallado la lectura:

```
1  try {
2      BufferedReader br = new BufferedReader(new FileReader("datos.txt"));
3      System.out.println(br.readLine());
4      br.close();
5  } catch (FileNotFoundException e) {
6      System.err.println("El fichero no existe: " + e.getMessage());
7  } catch (IOException e) {
8      System.err.println("Error de lectura: " + e.getMessage());
9  }
```

Bloque catch general y orden de los bloques

Además de gestionar excepciones concretas, se puede añadir un bloque que capture `Exception`. Como todas las excepciones heredan de ella, este bloque actúa como una **red de seguridad** para cualquier excepción que no hayan capturado los bloques anteriores:

```
try {
    BufferedReader br = new BufferedReader(new FileReader("datos.txt"));
    System.out.println(br.readLine());
    br.close();
} catch (FileNotFoundException e) {
    System.err.println("El fichero no existe: " + e.getMessage());
} catch (IOException e) {
    System.err.println("Error de lectura: " + e.getMessage());
}
```

Aquí entra en juego la **jerarquía de excepciones**. Un bloque `catch` captura su tipo y también todos sus subtipos: `catch (IOException e)` captura igualmente una `FileNotFoundException`, porque esta hereda de `IOException`; y `catch (Exception e)` lo captura prácticamente todo. Por eso los bloques deben ir ordenados **de lo más específico a lo más general**.

Y no es una simple recomendación: si colocas un tipo general antes que uno de sus subtipos, el programa **no compila**, porque el segundo bloque no se alcanzaría nunca. Este código, por ejemplo, es incorrecto:

```
try {
    BufferedReader br = new BufferedReader(new FileReader("datos.txt"));
    System.out.println(br.readLine());
    br.close();
} catch (IOException e) { // captura también FileNotFoundException
    System.err.println("Error de lectura: " + e.getMessage());
} catch (FileNotFoundException e) { // no compila: nunca se alcanzaría
    System.err.println("El fichero no existe: " + e.getMessage());
}
```

IntelliJ lo marca en rojo con el mensaje *Exception 'java.io.FileNotFoundException' has already been caught* (la excepción ya ha sido capturada). La solución es intercambiar los dos bloques, como en el ejemplo anterior.

Los bloques catch van de lo más específico a lo más general. Si el orden es incorrecto, el programa no compila.

Captura de varios tipos en un mismo bloque

Cuando varios tipos de excepción se gestionan exactamente igual, no hace falta repetir el bloque. Se pueden agrupar en un único `catch` separándolos con una barra vertical. Es lo que hacía el ejemplo de lectura con DOM del apartado 5.2:

```
1 } catch (ParserConfigurationException | SAXException | IOException e) {
2     System.err.println("Error al leer el XML: " + e.getMessage());
3 }
```

Si se produce cualquiera de las tres excepciones, se ejecuta el mismo bloque. Esta forma solo admite tipos que no hereden unos de otros: `FileNotFoundException` | `IOException`, por ejemplo, no compila, porque la primera ya está incluida en la segunda.

6.4 Declaración de excepciones lanzadas por un método

A veces, el método donde se produce una excepción no es el lugar adecuado para gestionarla. Un método que escribe en un fichero no sabe si, en caso de error, el programa debe mostrar un mensaje, reintentar o usar otro fichero: eso lo decide quien lo llama. Para estos casos, Java permite que el método **declare** la excepción y traslade la responsabilidad de gestionarla.

Uso de throws en la declaración de un método

La palabra reservada `throws` se escribe en la firma del método, seguida del tipo de excepción que puede lanzar. Con ella el método indica que no gestiona internamente esa excepción, sino que la delega al código que lo invoca. El siguiente método escribe un texto en un fichero y declara que puede lanzar una `IOException`:

```
import java.io.FileWriter;
import java.io.IOException;

public class EjemploThrows {
    public static void escribirArchivo(String contenido) throws IOException {
        FileWriter escritor = new FileWriter("archivo.txt");
        escritor.write(contenido);
        escritor.close(); // Puede lanzar IOException
    }
}
```

Como el método declara la excepción, no necesita ningún `try-catch` dentro. Ahora es el método que llama a `escribirArchivo` el responsable de gestionarla, capturándola en un bloque `try-catch` (o declarándola a su vez con `throws`, trasladándola un nivel más arriba). Esta clase va en un fichero aparte, `EjemploPrincipal.java`:

```
import java.io.IOException;

public class EjemploPrincipal {
    public static void main(String[] args) {
        try {
            EjemploThrows.escribirArchivo("Hola, mundo");
        } catch (IOException e) {
            System.err.println("Error al escribir: " + e.getMessage());
        }
    }
}
```

Si la escritura falla, la excepción sale de `escribirArchivo`, llega a `main` y la captura su bloque `catch`, que muestra el mensaje de error.

¿Cuándo capturar y cuándo declarar? Como criterio general, captura la excepción en el método que sabe qué hacer con ella, y declárala en los métodos que no lo saben. En los métodos auxiliares que leen o escriben ficheros suele ser mejor declarar; en `main`, que es el último punto de la cadena, conviene capturar para mostrar un mensaje claro en lugar de la traza de la pila.

Cierre de recursos: `try-with-resources`

El método `escribirArchivo` tiene el mismo problema que el ejemplo de lectura del apartado 6.2: si `write()` lanza una excepción, `close()` no se ejecuta y el fichero queda abierto. Un fichero abierto consume recursos del sistema operativo y, en Windows, impide que otros programas lo borren o lo modifiquen.

La solución clásica era el bloque `finally`, que se ejecuta siempre al salir del `try`, haya error o no. Pero como `close()` también puede lanzar una `IOException`, hay que meter otro `try` dentro del `finally`, y el código queda así de largo:

```
BufferedReader br = null;
try {
    br = new BufferedReader(new FileReader("datos.txt"));
    System.out.println(br.readLine());
} catch (IOException e) {
    System.err.println("Error al leer: " + e.getMessage());
} finally {
    if (br != null) {
        try {
            br.close();
        } catch (IOException e) {
            System.err.println("Error al cerrar: " + e.getMessage());
        }
    }
}
```

Desde Java 7 existe una forma mucho más sencilla y segura de hacer lo mismo: **`try-with-resources`**. El recurso se declara entre paréntesis justo después de la palabra `try`, y Java se encarga de cerrarlo automáticamente al salir del bloque:

```
try (BufferedReader br = new BufferedReader(new FileReader("datos.txt"))) {
    String linea;
    while ((linea = br.readLine()) != null) {
        System.out.println(linea);
    }
} catch (IOException e) {
    System.err.println("Error al leer: " + e.getMessage());
}
```

Sus reglas son pocas:

- El recurso se declara entre los paréntesis del `try`.
- Se cierra siempre al salir del bloque, tanto si termina con normalidad como si se produce una excepción. No hace falta llamar a `close()`.

- Se pueden declarar varios recursos separados por punto y coma, y se cierran en orden inverso al de su declaración.
- Solo funciona con clases que implementan la interfaz `AutoCloseable`. Todas las clases de flujos que has usado en esta unidad (`FileReader`, `BufferedReader`, `FileWriter`, `BufferedWriter`...) la implementan.

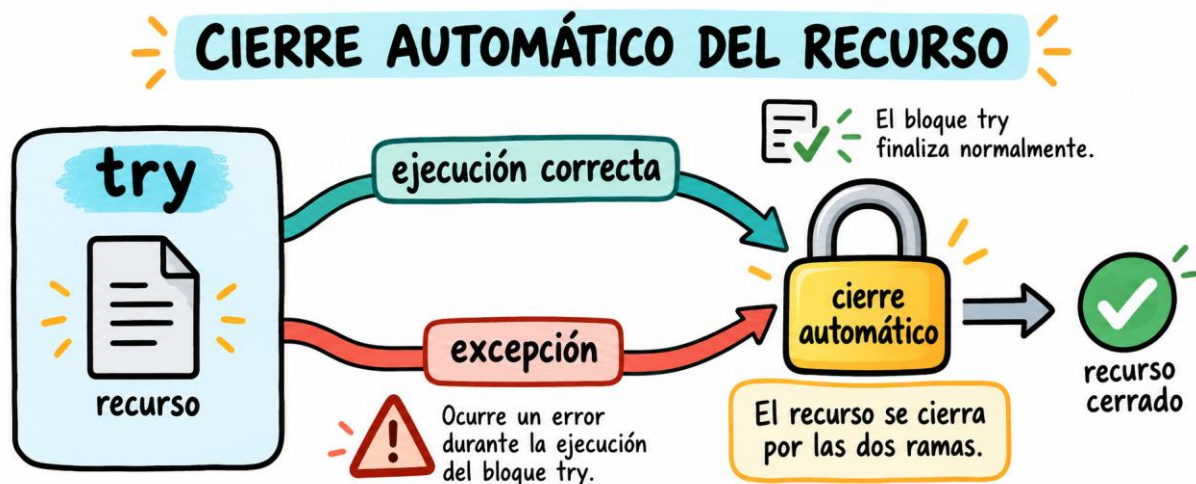
Este ejemplo copia un fichero línea a línea con dos recursos a la vez. Al salir del bloque se cierra primero `bw` y después `br`:

```
try (BufferedReader br = new BufferedReader(new FileReader("origen.txt"));
    BufferedWriter bw = new BufferedWriter(new FileWriter("copia.txt"))) {
    String linea;
    while ((linea = br.readLine()) != null) {
        bw.write(linea);
        bw.newLine();
    }
} catch (IOException e) {
    System.err.println("Error al copiar: " + e.getMessage());
}
```

Por último, `try-with-resources` también se combina con `throws`. Un bloque `try-with-resources` puede no tener `catch`, y entonces la excepción se propaga al método que llama, pero con la garantía de que el recurso se ha cerrado antes. Así queda el método `escribirArchivo` corregido:

```
public static void escribirArchivo(String contenido) throws IOException {
    try (FileWriter escritor = new FileWriter("archivo.txt")) {
        escritor.write(contenido);
    } // el fichero se cierra aquí, tanto si write() falla como si no
}
```

Si abres un fichero, ábrelo con `try-with-resources`: se cierra siempre, haya error o no.



7 Documentación de las aplicaciones

ECLIPSE	INTELIJ
	

8 Bibliografía

- [1] Cortijo Bon, Carlos A. (2019). Capítulo 7 “Bases de datos de XML”. En editorial Síntesis (España). ISBN: 978-84-9171-356-2. *Acceso a datos*.
- [2] Córcoles Tendero, Jose Eduardo. Capítulo 5 “Bases de datos XML”. En editorial RAMA (España). E-ISBN: 9788499643908
- [3] Camarena Estruch, Joan Gerard; Múrcia Andres, Jose A. Unidad 1: “Acceso a ficheros”. En editorial McGraw Hill. ISBN 9788448626730. *Acceso a datos*